

```
=====
```

TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance

```
=====
```

Model	Best LR	Val Loss	Macro P	Macro R
Macro F1 Time				
FFNN (Sequential IDs)	1e-04	0.0919	0.5516	0.5860
0.5624 00:00:39				
FFNN (Learnable Embeddings)	5e-05	0.0726	0.6516	0.6848
0.6662 00:00:38				

```
-----
```

1.4 Task 3 - Recursive Neural Network (RNN)

```
[ ]: # Create directory for plots
save_dir = results_path + 'images/' + 'task3_plots/'
os.makedirs(save_dir, exist_ok=True)

[ ]: # --- Choose number of epochs depending on device (GPU vs CPU) ---

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Use fewer epochs when running on CPU to avoid extremely long runs
if torch.cuda.is_available():
    num_epochs = 150
else:
    num_epochs = 15

print(f"Number of epochs: {num_epochs}")
```

Number of epochs: 150

1.4.1 Sequence Modeling with RNNs

```
[ ]: # --- Build vocabulary and API-to-ID mappings from training data ---

def build_vocab_from_train(df_train, seq_col="api_call_sequence"):
    """
    Extract the set of unique API calls from the training set only.
    Returns a sorted list of API call names (vocab).
    """
    vocab = sorted({api for seq in df_train[seq_col] for api in seq})
    return vocab

def build_api_mappings(vocab):
    """
    Create dictionaries mapping API names to integer IDs and vice versa.
```

```

Reserves indices 0 and 1 for <PAD> and <UNK> special tokens.
"""

api_to_id = {"<PAD>": PAD_IDX, "<UNK>": UNK_IDX}
for i, api in enumerate(vocab):
    api_to_id[api] = i + 2 # start after PAD/UNK
id_to_api = {i: api for api, i in api_to_id.items()}
return api_to_id, id_to_api

# Build vocabulary from training data only (no data leakage)
vocab_train = build_vocab_from_train(df_train)
api_to_id, id_to_api = build_api_mappings(vocab_train)

```

```

[ ]: # --- Encode API call sequences to integer IDs ---

def encode_sequences(df, api_to_id, seq_col="api_call_sequence",
    ↪label_col="is_malware", max_len=None):
    """
    Convert API call sequences to lists of integer IDs using the mapping.
    Unknown API calls (not seen in training) are mapped to <UNK> (PAD_IDX).
    Optionally truncate sequences to max_len (but we preserve full length for
    ↪RNNs).
    """

    seqs, labels = [], []
    for seq, y in zip(df[seq_col], df[label_col]):
        ids = [api_to_id.get(api, PAD_IDX) for api in seq]
        if max_len is not None:
            ids = ids[:max_len] # only if you explicitly choose to truncate
        seqs.append(ids)
        labels.append(int(y))
    return seqs, labels

# Encode training and test sequences (no truncation - RNNs handle variable
    ↪lengths)
train_seqs, train_labels = encode_sequences(df_train, api_to_id)
test_seqs, test_labels = encode_sequences(df_test, api_to_id)

```

```

[ ]: # --- Split training data into train/validation subsets ---

# Create indices for stratified train/val split (preserves class balance)
idx = list(range(len(train_seqs)))
train_idx, val_idx = train_test_split(
    idx, test_size=0.2, random_state=42, stratify=train_labels
)

def subset_by_idx(seqs, labels, indices):
    """Helper to extract sequences and labels at given indices."""
    return [seqs[i] for i in indices], [labels[i] for i in indices]

```

```
# Create training and validation subsets
train_seqs_sub, train_labels_sub = subset_by_idx(train_seqs, train_labels,
↳train_idx)
val_seqs_sub, val_labels_sub = subset_by_idx(train_seqs, train_labels, val_idx)
```

```
[ ]: # --- Define PyTorch Dataset for variable-length sequences ---
```

```
class SeqDataset(torch.utils.data.Dataset):
    """
    A PyTorch Dataset that wraps sequences and their labels.
    Each item returns a tensor of API IDs and a scalar label.
    Note: Sequences have variable lengths at this stage.
    """
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        x = torch.tensor(self.sequences[idx], dtype=torch.long)
        y = torch.tensor(self.labels[idx], dtype=torch.float32) # scalar
        return x, y
```

```
[ ]: # --- Custom collate function for padding and packing ---
```

```
def collate_fn(batch):
    """
    Custom collate function for DataLoader:
    1. Sorts sequences by length (descending) - required for
↳pack_padded_sequence
    2. Pads all sequences to the max length in this batch
    3. Returns padded sequences, original lengths, and labels (CPU tensors)
    Note: Device transfer is handled by the training loop, not the collate
↳function.
    """
    sequences, labels = zip(*batch)

    # Compute original lengths before padding
    lengths = torch.tensor([len(s) for s in sequences], dtype=torch.long)
    # Sort by length (descending) for pack_padded_sequence compatibility
    lengths_sorted, perm_idx = lengths.sort(descending=True)

    # Reorder sequences and labels according to sorted indices
    sequences = [sequences[i] for i in perm_idx]
```

```

labels = torch.stack([labels[i] for i in perm_idx]).unsqueeze(1) # (B,1)

# Pad sequences to max length in batch (dynamic padding)
padded = pad_sequence(sequences, batch_first=True, padding_value=PAD_IDX)

# Return CPU tensors - training loop handles device transfer
return padded, lengths_sorted, labels

```

```

[ ]: # --- Create DataLoaders with custom collate function ---

# Wrap sequences in Dataset objects
train_ds = SeqDataset(train_seqs_sub, train_labels_sub)
val_ds = SeqDataset(val_seqs_sub, val_labels_sub)
test_ds = SeqDataset(test_seqs, test_labels)

# Create DataLoaders with dynamic padding via collate_fn
# Note: collate_fn handles variable-length sequences by padding per batch
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True,
    ↪collate_fn=collate_fn)
val_loader = DataLoader(val_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)
test_loader = DataLoader(test_ds, batch_size=64, shuffle=False,
    ↪collate_fn=collate_fn)

```

Q: With RNNs, do you still have to pad your data? If yes, how? Yes, when working with sequences of varying lengths using standard PyTorch RNN layers (`nn.RNN`, `nn.LSTM`, `nn.GRU`), you **still need to pad the data**.

While RNNs can theoretically process sequences of any length, the requirement for processing data in batches on hardware like GPUs necessitates that all sequences within a single batch have the same dimensions. Padding ensures this uniform shape.

The padding is typically done by adding a special padding token (often mapped to index 0) to the end of shorter sequences until they match the length of the longest sequence in the current batch (or a predefined maximum length).

However, to prevent the RNN from processing the padding tokens as if they were real data, a technique called “**packing padded sequences**” is used. This is implemented using `torch.nn.utils.rnn.pack_padded_sequence`. This function takes the padded sequences and their original lengths, and effectively tells the RNN to only perform computations on the non-padded elements. After the RNN processes the packed sequence, the output can be “unpacked” back into a padded sequence using `torch.nn.utils.rnn.pad_packed_sequence`.

In the provided code, the `collate_fn` function handles this. It pads sequences to the maximum length within each batch and also sorts the sequences by length, which is a requirement for `pack_padded_sequence`. The `SimpleRNNClassifier` and `LSTMClassifier` models then use `pack_padded_sequence` within their forward methods to process the variable-length inputs efficiently while ignoring the padding.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case. When using RNNs with padding and packing (as implemented in Task 3), you **do not strictly *have* to truncate the testing sequences** if the model is designed to handle variable lengths via padding/packing.

Here's why:

1. **Padding and Packing:** The core mechanism used for handling variable-length sequences in the implemented RNN models is padding combined with `pack_padded_sequence`. This approach allows the RNN layers to process only the actual data in each sequence, effectively ignoring the padding tokens. The lengths of the original sequences are passed to `pack_padded_sequence`, which manages the computations for each time step up to the true length of each sequence in the batch.
2. **Variable Length Handling:** Because the packing/unpacking mechanism handles sequences of different lengths within a batch, the model can naturally handle test sequences that are longer or shorter than the training sequences (up to reasonable limits imposed by memory or potential vanishing/exploding gradients for extremely long sequences).
3. **Information Loss:** Truncating sequences, especially long ones, can lead to a loss of potentially important information that might be located at the end of the sequence. By using padding and packing, we avoid this information loss for sequences within the limits the model can handle.

In summary, while truncation was necessary for the fixed-input FFNN in Task 2, the padding and packing strategy employed for the RNNs in Task 3 allows the model to work with the full, variable-length sequences from the test set without needing to truncate them, thus retaining all available sequential information.

Unlike the FFNN in Task 2, which requires a fixed-size input vector due to its rigid weight matrix, RNNs do not require test sequence truncation. This is due to the temporal weight-sharing nature of the architecture: the same weights are applied at each time step, allowing the model to process sequences of any length. While we use padding and packing to manage batches efficiently and prevent 'zero-pollution' of the hidden state, the underlying RNN logic naturally accommodates the full API trace. By avoiding truncation, we ensure that malicious behaviors occurring late in a sequence are not discarded, preserving the full context of the software's execution.

```
[ ]: # Reusable training loop for RNN-like modules
def train_rnn_common(model, train_loader, val_loader, optimizer, criterion,
    epochs, min_delta=None, patience=None, scheduler=None):
    """
    A generic training loop for RNN-based models that supports early stopping.
    Now also tracks validation F1 macro score.
    """
    device = next(model.parameters()).device

    best_val = math.inf
    best_state = None
    trigger = 0
    history = {'train_loss': [], 'val_loss': [], 'val_f1_macro': []}
```

```

start_time = time.time()

for epoch in range(1, epochs+1):
    # --- Training Phase ---
    model.train()
    train_losses = []
    for X_batch, lengths, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch, lengths)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())
    train_loss = float(np.mean(train_losses)) if train_losses else 0.0

    # --- Validation Phase ---
    model.eval()
    val_losses = []
    all_preds = []
    all_targets = []
    with torch.no_grad():
        for X_batch, lengths, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch, lengths)
            loss = criterion(outputs, y_batch)
            val_losses.append(loss.item())

            # Collect predictions for F1 score
            probs = torch.sigmoid(outputs).view(-1)
            preds = (probs > 0.5).long()
            all_preds.extend(preds.cpu().tolist())
            all_targets.extend(y_batch.view(-1).cpu().tolist())

    val_loss = float(np.mean(val_losses)) if val_losses else 0.0
    val_f1_macro = f1_score(all_targets, all_preds, average='macro',
↪zero_division=0)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_f1_macro'].append(val_f1_macro)

    # --- Scheduler Step ---
    if scheduler is not None:
        last_lr = optimizer.param_groups[0]['lr']
        if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLRonPlateau):

```

```

        scheduler.step(val_loss)
    else:
        scheduler.step()
        curr_lr = optimizer.param_groups[0]['lr']
        if curr_lr != last_lr:
            print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to_
↪{curr_lr:.6f}")

        if (epoch % 5 == 0) or (epoch == 1) or (epoch == epochs):
            print(f'Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} -_
↪Val Loss: {val_loss:.4f} - Val F1: {val_f1_macro:.4f}')

        # --- Early Stopping ---
        if val_loss < best_val - (min_delta if min_delta is not None else 0):
            best_val = val_loss
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().
↪items()}
            trigger = 0
        else:
            trigger += 1
            if trigger >= patience:
                print(f'Early stopping at epoch {epoch} (best val loss_
↪{best_val:.6f})')
                break

    elapsed = time.time() - start_time

    if best_state is not None:
        model.load_state_dict(best_state)

    return model, history, elapsed

```

```

[ ]: # Prediction helper (binary classification threshold)
def predict_loader(model, loader):
    """
    Run the model on a DataLoader and return integer arrays for
    ground-truth labels and predicted labels (0/1).
    """
    # Set model to evaluation mode (e.g., disables dropout)
    model.eval()

    # Detect device from model parameters
    device = next(model.parameters()).device

    ys_true = [] # To store all true labels
    ys_pred = [] # To store all predicted labels

```

```

# Disable gradient calculations to save memory and computation
with torch.no_grad():
    # Iterate over all batches in the data loader
    for X_batch, lengths, y_batch in loader:
        # Move batch to device
        X_batch = X_batch.to(device)

        # Get model outputs (logits)
        out = model(X_batch, lengths)

        # Convert logits to probabilities using sigmoid
        probs = torch.sigmoid(out).cpu().numpy().ravel()

        # Apply threshold for binary classification
        preds = (probs >= 0.5).astype(int)

        # Store the ground-truth and predicted labels from this batch
        ys_true.extend(y_batch.cpu().numpy().ravel().astype(int).tolist())
        ys_pred.extend(preds.tolist())

# Convert the lists of all labels/preds into final numpy arrays
return np.array(ys_true, dtype=int), np.array(ys_pred, dtype=int)

```

```

[ ]: def plot_cm_task3(model, loader, model_name, save_dir=None):
    """
    Specific for RNNs. Uses your existing 'predict_loader' logic.
    """
    # Use the helper function you already defined in the notebook
    y_true, y_pred = predict_loader(model, loader)

    # Plot
    _internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[ ]: # Function to train a model with a specific learning rate
def train_with_lr(model_class, model_params, initial_lr, train_loader,
    ↪ val_loader, criterion, num_epochs, min_delta, patience, scheduler_params,
    ↪ device):
    """
    Train a model with a specific initial learning rate and return the history.
    """
    # Create a fresh instance of the model
    model = model_class(**model_params).to(device)

    # Create optimizer with the specified learning rate
    optimizer = optim.AdamW(model.parameters(), lr=initial_lr)

    # Create scheduler

```



```

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    **scheduler_params
)

# Train the model
model, history, elapsed = train_rnn_common(
    model,
    train_loader,
    val_loader,
    optimizer,
    criterion,
    epochs=num_epochs,
    min_delta=min_delta,
    patience=patience,
    scheduler=scheduler
)

return model, history, elapsed

```

Q: Is the RNN padding more memory efficient compared to the FFNN's one? Why? Yes, the padding method used with RNNs, particularly when combined with `pack_padded_sequence`, is generally **more memory efficient** than the padding required for the FFNN in Task 2. Here's the key difference:

1. **FFNN Padding (Task 2):** For the FFNN, each sequence needed to be padded or truncated to a *single, fixed maximum length* to create a uniform input tensor. This meant that even short sequences would occupy the same large memory footprint as the longest sequences after padding. This can be memory-intensive, especially when the distribution of sequence lengths is wide and the chosen fixed length is large (like the maximum length).
2. **RNN Padding with Packing (Task 3):** With RNNs and `pack_padded_sequence`, padding is still used to create batches of uniform shape, but the `pack_padded_sequence` function allows the RNN to effectively ignore the padded elements during computation. The RNN processes the actual data points sequentially based on their original lengths. This means that while the padded tensor exists, the *computations* and the resulting hidden states primarily reflect the actual length of each sequence, leading to more efficient memory usage during the core processing, especially within the RNN layers themselves.

In summary, the FFNN approach forces all inputs to the same large, fixed size, leading to potentially wasted memory for shorter sequences. The RNN with packing, while still using padding for batching, is more efficient in how it processes and uses memory for variable-length inputs by focusing computation only on the real data points.

```

[ ]: # --- Variation 1: Simple one-directional RNN (nn.RNN) ---

class SimpleRNNClassifier(nn.Module):

```

```

def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
↳num_classes=1, bidirectional=False, dropout=0.25, padding=0):
    super(SimpleRNNClassifier, self).__init__()

    # Embedding layer: Maps vocabulary indices (integers) to dense vectors
↳(embeddings).
    # padding_idx=0 ensures the padding token doesn't contribute to
↳gradients.
    self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,
↳padding_idx=padding)

    # Simple RNN layer: Processes the embedded sequences.
    self.rnn = nn.RNN(input_size=embed_dim, hidden_size=hidden_size,
↳num_layers=num_layers,
                        batch_first=True, nonlinearity='tanh',
↳bidirectional=bidirectional)

    self.bidirectional = bidirectional
    fc_in = hidden_size * (2 if bidirectional else 1)

    # Final fully connected layer for classification.
    self.fc = nn.Linear(fc_in, num_classes)
    self.dropout = nn.Dropout(dropout)

def forward(self, x, lengths):
    # Embed the input indices
    emb = self.embedding(x) # Shape: (batch_size, max_seq_len, embed_dim)
    emb = self.dropout(emb)

    # Pack the sequence to let the RNN ignore padding
    packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
↳enforce_sorted=True)

    # Process the packed sequence through the RNN
    packed_out, hn = self.rnn(packed) # hn is the final hidden state(s)

    # hn shape: (num_layers * num_directions, batch_size, hidden_size)

    # Get the final hidden state to represent the whole sequence.
    if self.bidirectional:
        # Concatenate the final forward (hn[-2]) and backward (hn[-1])
↳states
        h_last = torch.cat([hn[-2], hn[-1]], dim=1)
    else:
        # Use the final hidden state from the last layer (hn[-1])
        h_last = hn[-1]

```

```

    # Pass the final hidden state through the classifier
    out = self.fc(h_last)
    return out

```

```

[ ]: # --- Variation 1: Simple Unidirectional RNN ---

print("\n" + "=" * 80)
print("Training Simple RNN with different learning rates")
print("=" * 80)

# Define learning rates to test
learning_rates = [2e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
vocab_size_rnn = len(api_to_id)
simple_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': False,
    'dropout': 0.3
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Compute class balance from training subset and set pos_weight for
↳ BCEWithLogitsLoss
pos = sum(train_labels_sub) # number of positive (1) samples
neg = len(train_labels_sub) - pos

# Compute weights
pos_weight = compute_pos_weight(train_labels_sub).to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# Setup Scheduler
scheduler_params_simple = {
    'mode': 'min', # reduce when metric stops decreasing (val_loss)
    'factor': 0.5, # halve the LR when triggered
    'patience': 8, # wait 8 epochs with no improvement before cutting LR
    'min_lr': 1e-7
}

```

```

# Store histories and models for each LR
histories_simple = {}
models_simple = {}
elapsed_simple = {}

for lr in learning_rates:
    print(f"\n--- Training Simple RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNClassifier,
        model_params=simple_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_simple,
        device=device
    )
    histories_simple[lr] = history
    models_simple[lr] = model
    elapsed_simple[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
    val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training Simple RNN with different learning rates
=====

```

```

Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

```

```

--- Training Simple RNN with lr=2e-04 ---
Epoch 1/150 - Train Loss: 0.0981 - Val Loss: 0.0868 - Val F1: 0.6071
Epoch 5/150 - Train Loss: 0.0794 - Val Loss: 0.0846 - Val F1: 0.6085
Epoch 10/150 - Train Loss: 0.0706 - Val Loss: 0.0763 - Val F1: 0.6637
Epoch 15/150 - Train Loss: 0.0663 - Val Loss: 0.0742 - Val F1: 0.6421
Epoch 20/150 - Train Loss: 0.0597 - Val Loss: 0.0675 - Val F1: 0.6778
Epoch 25/150 - Train Loss: 0.0539 - Val Loss: 0.0660 - Val F1: 0.6995
Epoch 30/150 - Train Loss: 0.0469 - Val Loss: 0.0560 - Val F1: 0.7323
Epoch 35/150 - Train Loss: 0.0406 - Val Loss: 0.0575 - Val F1: 0.6945
Epoch 40/150 - Train Loss: 0.0362 - Val Loss: 0.0555 - Val F1: 0.7260
Epoch 42: LR reduced from 0.000200 to 0.000100
Epoch 45/150 - Train Loss: 0.0312 - Val Loss: 0.0611 - Val F1: 0.7721
Epoch 50/150 - Train Loss: 0.0278 - Val Loss: 0.0614 - Val F1: 0.7782

```

Epoch 51: LR reduced from 0.000100 to 0.000050
Early stopping at epoch 53 (best val loss 0.053802)
Completed. Training time: 00:02:49 (169.90s) - Final val loss: 0.0576

--- Training Simple RNN with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1020 - Val Loss: 0.0876 - Val F1: 0.5383
Epoch 5/150 - Train Loss: 0.0792 - Val Loss: 0.0785 - Val F1: 0.6036
Epoch 10/150 - Train Loss: 0.0723 - Val Loss: 0.0748 - Val F1: 0.6283
Epoch 15/150 - Train Loss: 0.0674 - Val Loss: 0.0769 - Val F1: 0.6237
Epoch 20/150 - Train Loss: 0.0662 - Val Loss: 0.0680 - Val F1: 0.6622
Epoch 25/150 - Train Loss: 0.0611 - Val Loss: 0.0698 - Val F1: 0.6292
Epoch 30/150 - Train Loss: 0.0569 - Val Loss: 0.0638 - Val F1: 0.7067
Epoch 35/150 - Train Loss: 0.0553 - Val Loss: 0.0635 - Val F1: 0.6646
Epoch 40/150 - Train Loss: 0.0536 - Val Loss: 0.0619 - Val F1: 0.6806
Epoch 45/150 - Train Loss: 0.0498 - Val Loss: 0.0643 - Val F1: 0.7070
Epoch 50/150 - Train Loss: 0.0491 - Val Loss: 0.0620 - Val F1: 0.7228
Epoch 55/150 - Train Loss: 0.0465 - Val Loss: 0.0618 - Val F1: 0.6948
Epoch 57: LR reduced from 0.000100 to 0.000050
Epoch 60/150 - Train Loss: 0.0398 - Val Loss: 0.0585 - Val F1: 0.7256
Epoch 65/150 - Train Loss: 0.0407 - Val Loss: 0.0582 - Val F1: 0.6960
Epoch 70/150 - Train Loss: 0.0399 - Val Loss: 0.0569 - Val F1: 0.7027
Epoch 75/150 - Train Loss: 0.0362 - Val Loss: 0.0573 - Val F1: 0.7035
Epoch 80/150 - Train Loss: 0.0351 - Val Loss: 0.0583 - Val F1: 0.6963
Epoch 81: LR reduced from 0.000050 to 0.000025
Epoch 85/150 - Train Loss: 0.0341 - Val Loss: 0.0560 - Val F1: 0.7112
Epoch 90/150 - Train Loss: 0.0314 - Val Loss: 0.0570 - Val F1: 0.7510
Epoch 95/150 - Train Loss: 0.0306 - Val Loss: 0.0559 - Val F1: 0.7132
Epoch 100/150 - Train Loss: 0.0300 - Val Loss: 0.0596 - Val F1: 0.6859
Epoch 105/150 - Train Loss: 0.0319 - Val Loss: 0.0554 - Val F1: 0.6987
Epoch 107: LR reduced from 0.000025 to 0.000013
Epoch 110/150 - Train Loss: 0.0287 - Val Loss: 0.0538 - Val F1: 0.7200
Epoch 115/150 - Train Loss: 0.0292 - Val Loss: 0.0538 - Val F1: 0.7178
Epoch 120/150 - Train Loss: 0.0291 - Val Loss: 0.0533 - Val F1: 0.7192
Epoch 125/150 - Train Loss: 0.0282 - Val Loss: 0.0531 - Val F1: 0.7185
Epoch 130/150 - Train Loss: 0.0275 - Val Loss: 0.0530 - Val F1: 0.7215
Epoch 133: LR reduced from 0.000013 to 0.000006
Epoch 135/150 - Train Loss: 0.0260 - Val Loss: 0.0560 - Val F1: 0.7109
Epoch 140/150 - Train Loss: 0.0271 - Val Loss: 0.0559 - Val F1: 0.7185
Epoch 145/150 - Train Loss: 0.0278 - Val Loss: 0.0534 - Val F1: 0.7133
Epoch 150/150 - Train Loss: 0.0264 - Val Loss: 0.0549 - Val F1: 0.7171
Completed. Training time: 00:07:12 (432.78s) - Final val loss: 0.0549

--- Training Simple RNN with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1070 - Val Loss: 0.0870 - Val F1: 0.5701
Epoch 5/150 - Train Loss: 0.0793 - Val Loss: 0.0797 - Val F1: 0.6358
Epoch 10/150 - Train Loss: 0.0688 - Val Loss: 0.0766 - Val F1: 0.6251
Epoch 15/150 - Train Loss: 0.0663 - Val Loss: 0.0687 - Val F1: 0.6465
Epoch 20/150 - Train Loss: 0.0621 - Val Loss: 0.0696 - Val F1: 0.6836

Epoch 25/150 - Train Loss: 0.0600 - Val Loss: 0.0726 - Val F1: 0.6334
 Epoch 30/150 - Train Loss: 0.0563 - Val Loss: 0.0703 - Val F1: 0.6458
 Epoch 35/150 - Train Loss: 0.0541 - Val Loss: 0.0682 - Val F1: 0.6465
 Epoch 40/150 - Train Loss: 0.0505 - Val Loss: 0.0582 - Val F1: 0.7032
 Epoch 45/150 - Train Loss: 0.0473 - Val Loss: 0.0635 - Val F1: 0.7321
 Epoch 50/150 - Train Loss: 0.0467 - Val Loss: 0.0578 - Val F1: 0.6911
 Epoch 55/150 - Train Loss: 0.0431 - Val Loss: 0.0523 - Val F1: 0.7204
 Epoch 60/150 - Train Loss: 0.0436 - Val Loss: 0.0508 - Val F1: 0.7452
 Epoch 65/150 - Train Loss: 0.0386 - Val Loss: 0.0468 - Val F1: 0.7519
 Epoch 70/150 - Train Loss: 0.0352 - Val Loss: 0.0457 - Val F1: 0.7943
 Epoch 75/150 - Train Loss: 0.0369 - Val Loss: 0.0450 - Val F1: 0.8010
 Epoch 80/150 - Train Loss: 0.0320 - Val Loss: 0.0423 - Val F1: 0.7824
 Epoch 85/150 - Train Loss: 0.0312 - Val Loss: 0.0461 - Val F1: 0.7615
 Epoch 89: LR reduced from 0.000080 to 0.000040
 Epoch 90/150 - Train Loss: 0.0284 - Val Loss: 0.0435 - Val F1: 0.7789
 Epoch 95/150 - Train Loss: 0.0269 - Val Loss: 0.0469 - Val F1: 0.8096
 Epoch 98: LR reduced from 0.000040 to 0.000020
 Epoch 100/150 - Train Loss: 0.0256 - Val Loss: 0.0442 - Val F1: 0.7847
 Early stopping at epoch 100 (best val loss 0.042281)
 Completed. Training time: 00:04:45 (285.73s) - Final val loss: 0.0442

--- Training Simple RNN with lr=5e-05 ---

Epoch 1/150 - Train Loss: 0.1119 - Val Loss: 0.0912 - Val F1: 0.4903
 Epoch 5/150 - Train Loss: 0.0828 - Val Loss: 0.0803 - Val F1: 0.6229
 Epoch 10/150 - Train Loss: 0.0763 - Val Loss: 0.0740 - Val F1: 0.6495
 Epoch 15/150 - Train Loss: 0.0714 - Val Loss: 0.0704 - Val F1: 0.6881
 Epoch 20/150 - Train Loss: 0.0692 - Val Loss: 0.0733 - Val F1: 0.6593
 Epoch 25/150 - Train Loss: 0.0645 - Val Loss: 0.0691 - Val F1: 0.6513
 Epoch 27: LR reduced from 0.000050 to 0.000025
 Epoch 30/150 - Train Loss: 0.0631 - Val Loss: 0.0708 - Val F1: 0.6627
 Epoch 35/150 - Train Loss: 0.0632 - Val Loss: 0.0655 - Val F1: 0.6792
 Epoch 40/150 - Train Loss: 0.0618 - Val Loss: 0.0664 - Val F1: 0.6741
 Epoch 45/150 - Train Loss: 0.0604 - Val Loss: 0.0650 - Val F1: 0.6813
 Epoch 50/150 - Train Loss: 0.0595 - Val Loss: 0.0654 - Val F1: 0.6731
 Epoch 55/150 - Train Loss: 0.0562 - Val Loss: 0.0632 - Val F1: 0.7024
 Epoch 60/150 - Train Loss: 0.0563 - Val Loss: 0.0618 - Val F1: 0.6877
 Epoch 65/150 - Train Loss: 0.0528 - Val Loss: 0.0607 - Val F1: 0.7007
 Epoch 70/150 - Train Loss: 0.0519 - Val Loss: 0.0703 - Val F1: 0.6904
 Epoch 71: LR reduced from 0.000025 to 0.000013
 Epoch 75/150 - Train Loss: 0.0499 - Val Loss: 0.0626 - Val F1: 0.7120
 Epoch 80: LR reduced from 0.000013 to 0.000006
 Epoch 80/150 - Train Loss: 0.0498 - Val Loss: 0.0646 - Val F1: 0.7083
 Epoch 85/150 - Train Loss: 0.0487 - Val Loss: 0.0556 - Val F1: 0.7318
 Epoch 90/150 - Train Loss: 0.0468 - Val Loss: 0.0589 - Val F1: 0.7338
 Epoch 94: LR reduced from 0.000006 to 0.000003
 Epoch 95/150 - Train Loss: 0.0490 - Val Loss: 0.0574 - Val F1: 0.7166
 Epoch 100/150 - Train Loss: 0.0479 - Val Loss: 0.0571 - Val F1: 0.7152
 Epoch 103: LR reduced from 0.000003 to 0.000002

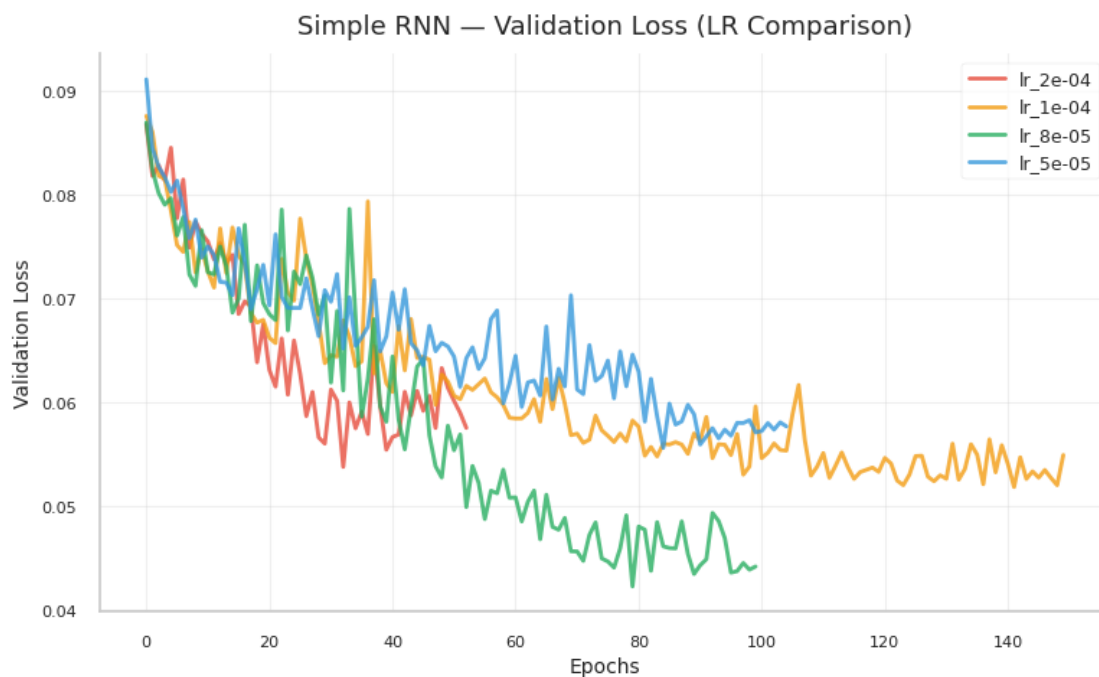
Epoch 105/150 - Train Loss: 0.0481 - Val Loss: 0.0577 - Val F1: 0.7274
 Early stopping at epoch 105 (best val loss 0.055634)
 Completed. Training time: 00:05:04 (304.40s) - Final val loss: 0.0577

```
[ ]: # --- Plot loss curves ---

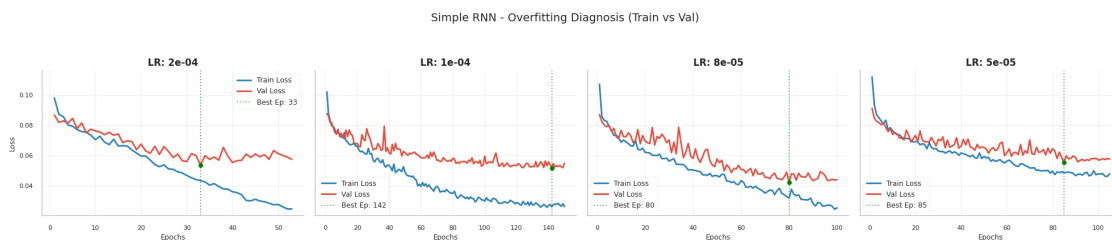
# Plot comparison
plot_lr_comparison(histories_simple, "Simple RNN", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_simple, "Simple RNN", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_lr_comparison.png

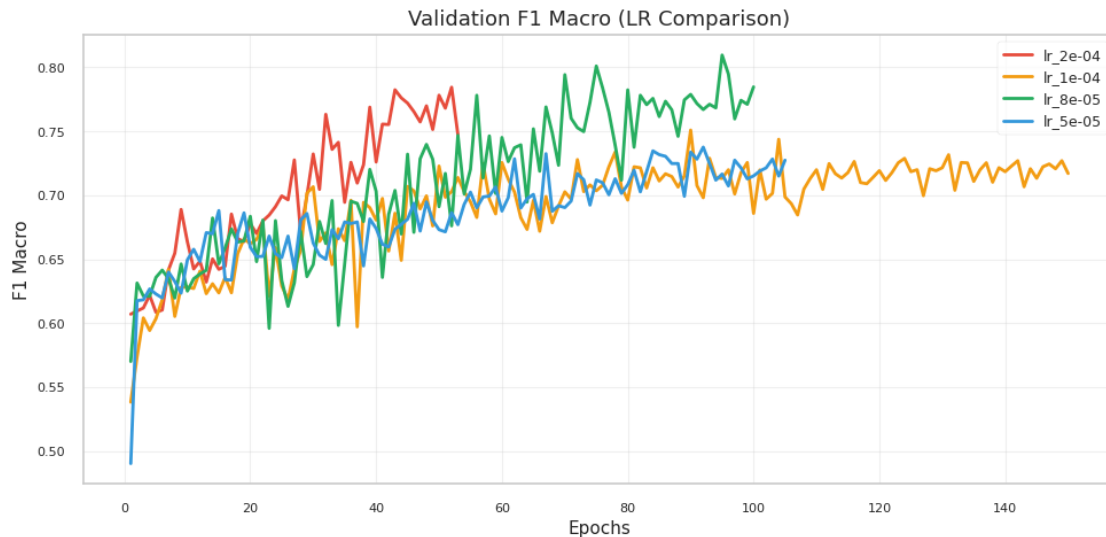


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/simple_rnn_diagnosis.png



```
[ ]: # --- Plot Validation F1 Macro for Simple RNN ---
plot_training_metrics(histories_seq, save_dir=save_dir,
↳plot_name="simple_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/comparison_f1_macro.png



```
[ ]: # --- Evaluate validation and test sets for ALL learning rates ---
print("\n" + "=" * 80)
print("Evaluation for ALL Simple RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]

    print(f"\n{'='*60}")
    print(f"Simple RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Simple RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
↳zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
```



```

print(f'Test report (Simple RNN - lr={lr:.0e}):')
print(classification_report(y_test_true, y_test_pred, digits=4,
↪zero_division=0))

# Find and highlight best model
best_lr_simple = min(histories_simple.items(), key=lambda x:
↪min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best Simple RNN - LR: {best_lr_simple[0]:.0e} (Min Val Loss:
↪{min(best_lr_simple[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

=====

Evaluation for ALL Simple RNN learning rates

=====

=====

Simple RNN with lr=2e-04

=====

Validation report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.4348	0.5738	0.4947	122
1	0.9832	0.9710	0.9771	3143
accuracy			0.9562	3265
macro avg	0.7090	0.7724	0.7359	3265
weighted avg	0.9628	0.9562	0.9591	3265

Test report (Simple RNN - lr=2e-04):

	precision	recall	f1-score	support
0	0.3978	0.5844	0.4733	243
1	0.9836	0.9657	0.9745	6262
accuracy			0.9514	6505
macro avg	0.6907	0.7750	0.7239	6505
weighted avg	0.9617	0.9514	0.9558	6505

=====

Simple RNN with lr=1e-04

=====

Validation report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.3785	0.6639	0.4821	122
1	0.9866	0.9577	0.9719	3143
accuracy			0.9467	3265
macro avg	0.6825	0.8108	0.7270	3265
weighted avg	0.9638	0.9467	0.9536	3265

Test report (Simple RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.3706	0.6543	0.4732	243
1	0.9862	0.9569	0.9713	6262
accuracy			0.9456	6505
macro avg	0.6784	0.8056	0.7223	6505
weighted avg	0.9632	0.9456	0.9527	6505

=====

Simple RNN with lr=8e-05

=====

Validation report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4703	0.7787	0.5864	122
1	0.9912	0.9660	0.9784	3143
accuracy			0.9590	3265
macro avg	0.7307	0.8723	0.7824	3265
weighted avg	0.9717	0.9590	0.9638	3265

Test report (Simple RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4257	0.7078	0.5317	243
1	0.9884	0.9630	0.9755	6262
accuracy			0.9534	6505
macro avg	0.7071	0.8354	0.7536	6505
weighted avg	0.9673	0.9534	0.9589	6505

=====

Simple RNN with lr=5e-05

=====

Validation report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.4088	0.6066	0.4884	122
1	0.9844	0.9660	0.9751	3143

accuracy			0.9525	3265
macro avg	0.6966	0.7863	0.7318	3265
weighted avg	0.9629	0.9525	0.9569	3265

Test report (Simple RNN - lr=5e-05):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.3708	0.5432	0.4407	243
1	0.9819	0.9642	0.9730	6262

accuracy			0.9485	6505
macro avg	0.6764	0.7537	0.7069	6505
weighted avg	0.9591	0.9485	0.9531	6505

=====
Best Simple RNN - LR: 8e-05 (Min Val Loss: 0.0423)
=====

```
[ ]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

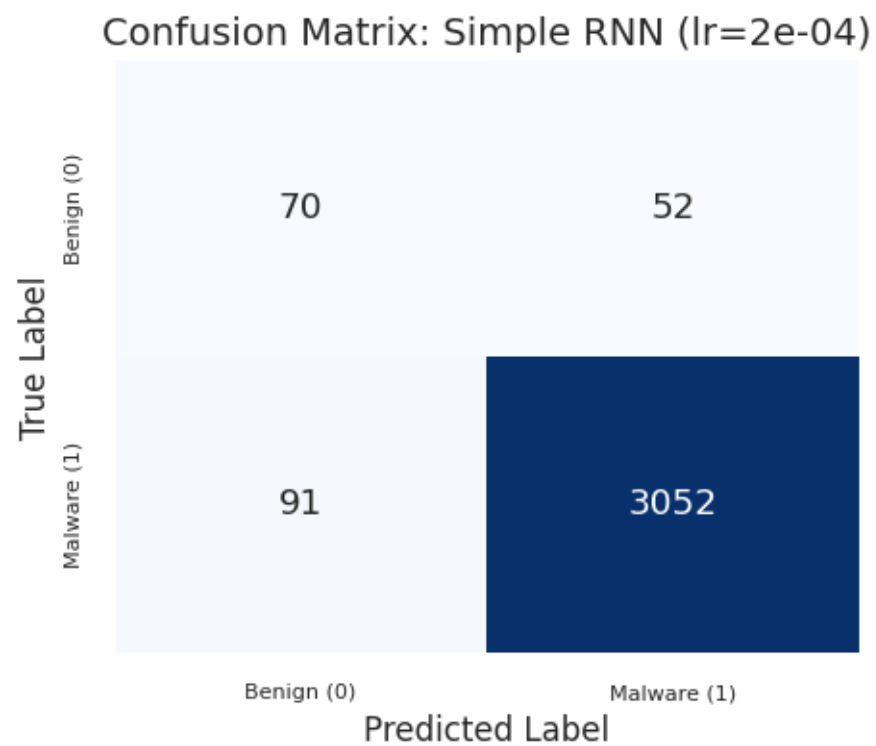
print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Simple RNN_
↪(lr={lr:.0e})", save_dir=save_dir)
```

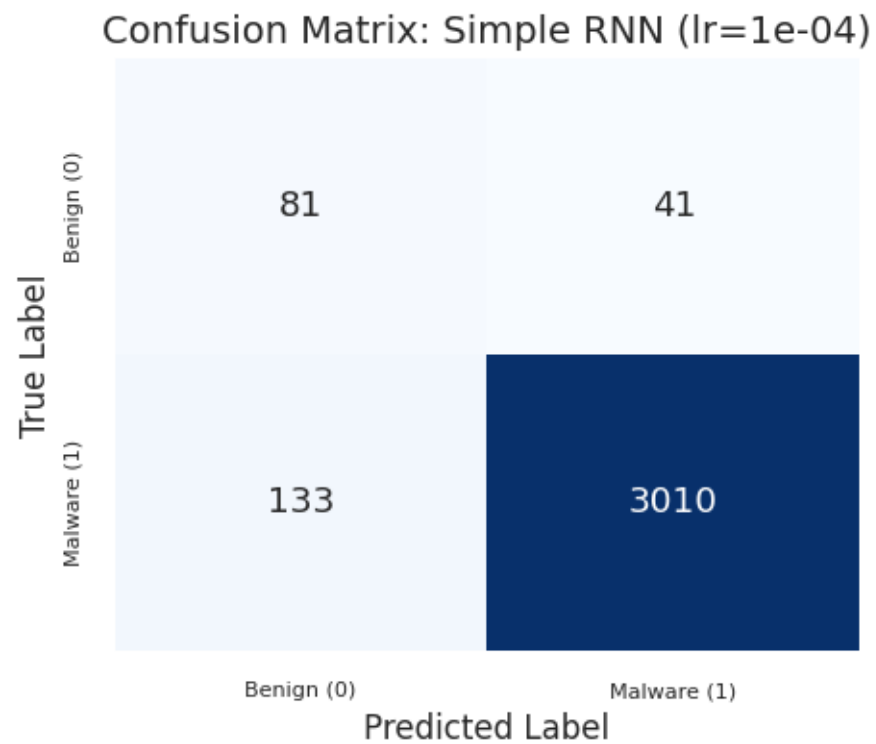
=====
Confusion matrices for ALL Simple RNN learning rates - VALIDATION
=====

Generating confusion matrix for Simple RNN with lr=2e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png



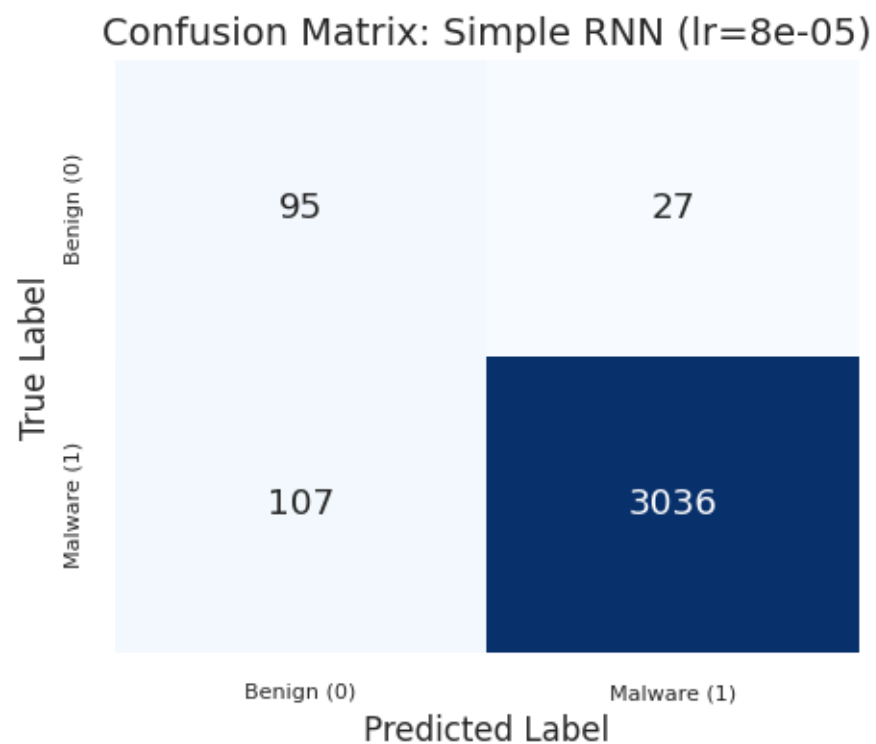
Generating confusion matrix for Simple RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=1e_04).png



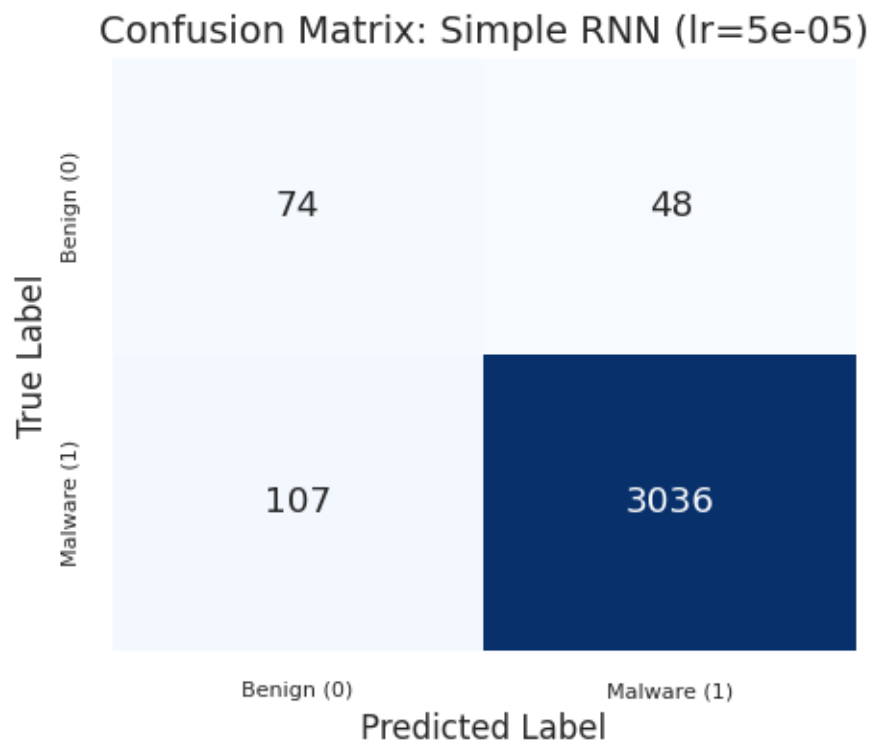
Generating confusion matrix for Simple RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



```
[ ]: # --- Plot confusion matrices for ALL learning rates - TEST ---

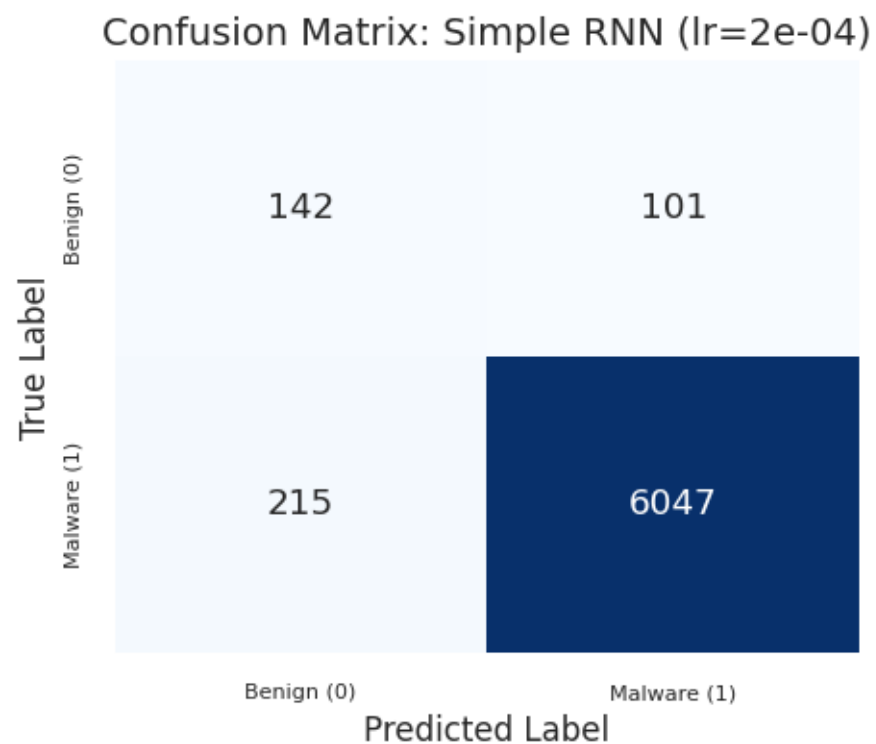
print("\n" + "=" * 80)
print("Confusion matrices for ALL Simple RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_simple[lr]
    print(f"\nGenerating confusion matrix for Simple RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Simple RNN_
↵(lr={lr:.0e})", save_dir=save_dir)
```

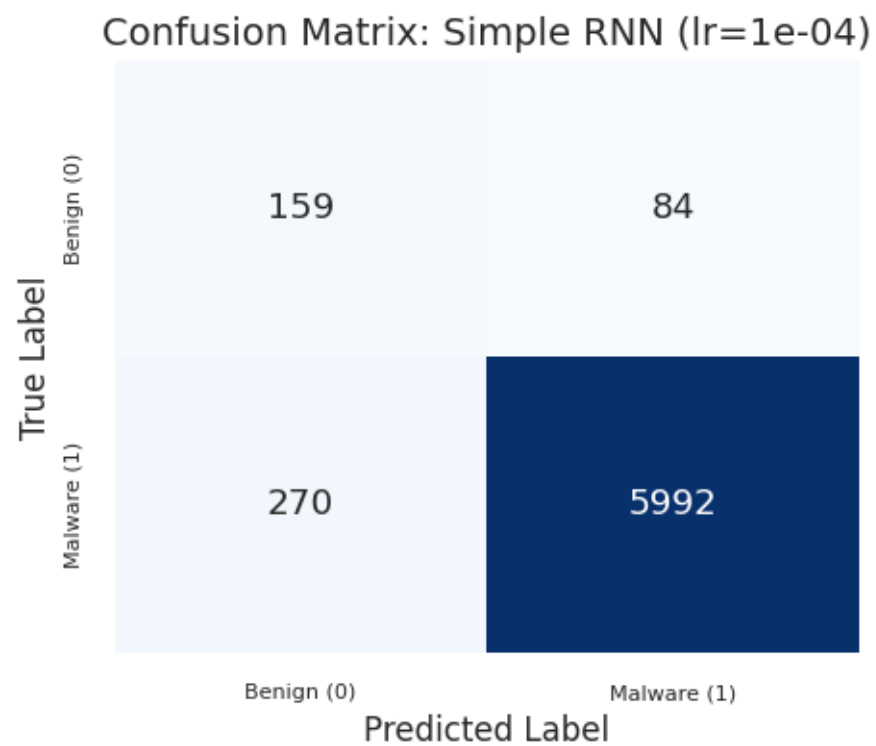
```
=====
Confusion matrices for ALL Simple RNN learning rates - TEST
=====
```

```
Generating confusion matrix for Simple RNN with lr=2e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=2e_04).png
```



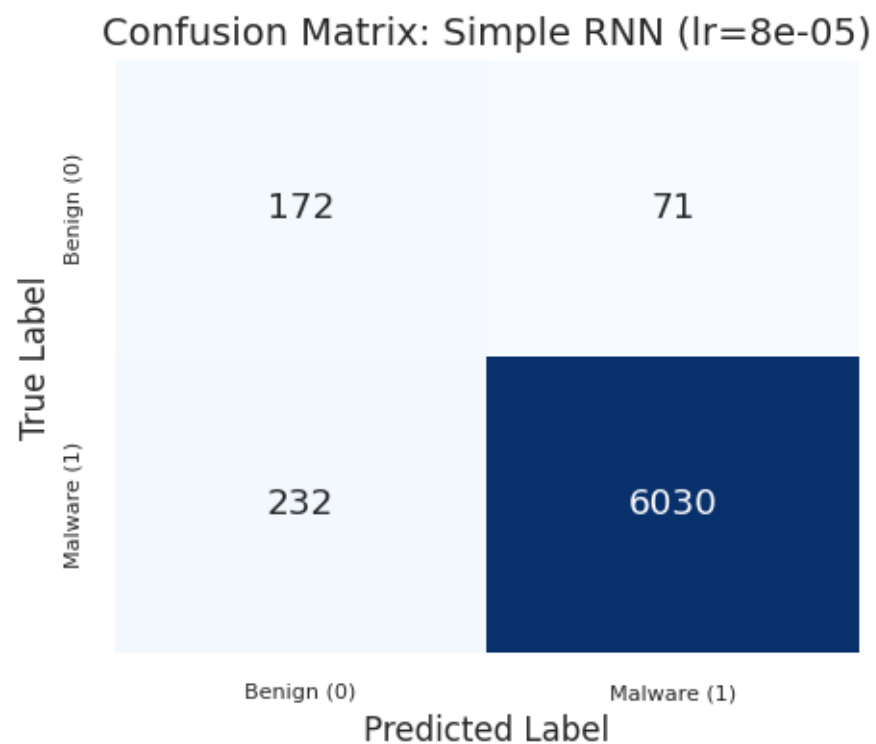
Generating confusion matrix for Simple RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_simple_rnn_(lr=1e_04).png



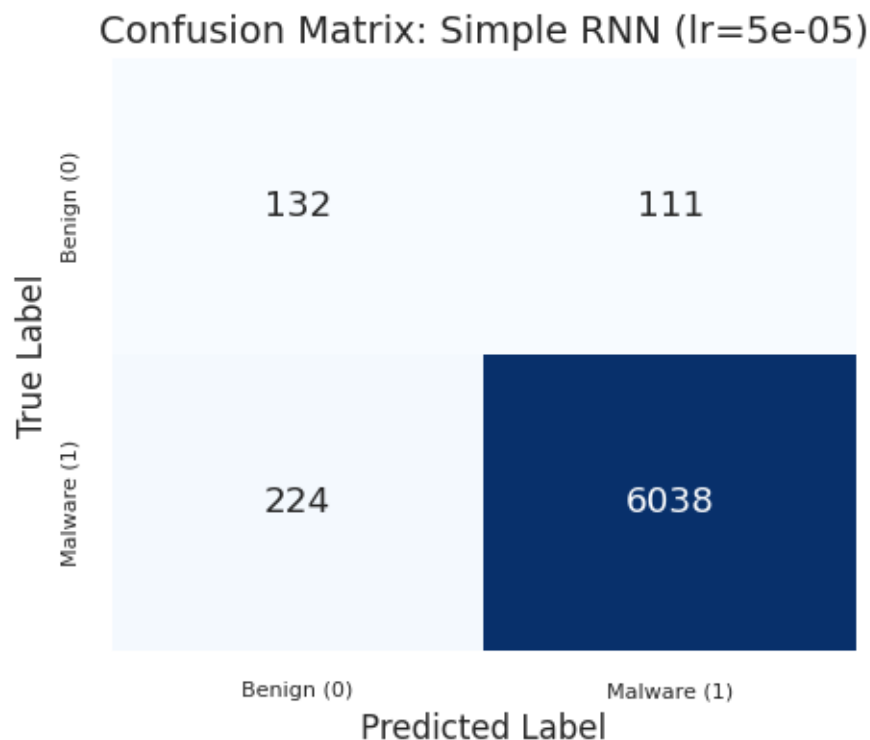
Generating confusion matrix for Simple RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=8e_05).png



Generating confusion matrix for Simple RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_simple_rnn_(lr=5e_05).png



Q: Start with a simple one-directional RNN. Is your network as fast as the FFNN? If not, why? Based on the training times recorded in the notebook:

- **FFNN (Sequential IDs)** training time: 00:01:24 — 84.52s seconds
- **FFNN (Learnable Embeddings)** training time: 00:01:36 96.88s— 96.88 seconds
- **Simple RNN** training time: 00:05:04 — 304 seconds

The simple one-directional RNN is **significantly slower** than both FFNN variations.

Here's why RNNs are typically slower than FFNNs for processing sequences:

1. **Sequential Processing:** RNNs process sequences step-by-step, one element at a time. The computation for the current time step depends on the output of the previous time step. This inherent sequential nature makes it difficult to parallelize computations across the time dimension, which is a major bottleneck, especially for long sequences.
2. **FFNN Parallelization:** FFNNs, on the other hand, process the entire input vector (the padded sequence in our case) in a single pass through its layers. The computations within each layer are independent across the elements of the input and can be highly parallelized across the batch and feature dimensions, making them much faster on hardware like GPUs.
3. **Backpropagation Through Time (BPTT):** During the training phase, RNNs utilize BPTT to update weights. This requires storing all intermediate hidden states for each time step to compute gradients through the entire chain of the sequence. This process is not only computationally expensive but also significantly more memory-intensive than the standard backpropagation used in FFNNs.

Even with techniques like `pack_padded_sequence` to optimize RNN training by avoiding computation on padding, the fundamental sequential dependency limits the extent of parallelization possible compared to the dense matrix multiplications in an FFNN.

1.4.2 Network Variations

```
[ ]: # --- Variation 2: Bidirectional RNN ---

print("\n" + "=" * 80)
print("Training Bidirectional RNN with different learning rates")
print("=" * 80)

# Define learning rates to test
learning_rates = [3e-04, 1e-04, 8e-05, 5e-05]

# Model parameters
bi_rnn_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.35
}

# Initialize early stopping parameters
min_delta = 0.00001
patience = 20

# Scheduler parameters
scheduler_params_bi = {
    'mode': 'min',
    'factor': 0.7,
    'patience': 7,
    'min_lr': 1e-7
}

# Store histories and models for each LR
histories_bi = {}
models_bi = {}
elapsed_bi = {}

for lr in learning_rates:
    print(f"\n--- Training Bi-RNN with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=SimpleRNClassifier,
```

```

        model_params=bi_rnn_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_bi,
        device=device
    )
    histories_bi[lr] = history
    models_bi[lr] = model
    elapsed_bi[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
↪val loss: {history['val_loss'][-1]:.4f}")

```

=====

Training Bidirectional RNN with different learning rates

=====

--- Training Bi-RNN with lr=3e-04 ---

```

Epoch 1/150 - Train Loss: 0.0963 - Val Loss: 0.0879 - Val F1: 0.5990
Epoch 5/150 - Train Loss: 0.0717 - Val Loss: 0.0782 - Val F1: 0.6398
Epoch 10/150 - Train Loss: 0.0631 - Val Loss: 0.0677 - Val F1: 0.6681
Epoch 15/150 - Train Loss: 0.0559 - Val Loss: 0.0681 - Val F1: 0.6655
Epoch 20/150 - Train Loss: 0.0474 - Val Loss: 0.0679 - Val F1: 0.7176
Epoch 25/150 - Train Loss: 0.0426 - Val Loss: 0.0630 - Val F1: 0.7010
Epoch 30/150 - Train Loss: 0.0350 - Val Loss: 0.0708 - Val F1: 0.7266
Epoch 34: LR reduced from 0.000300 to 0.000210
Epoch 35/150 - Train Loss: 0.0292 - Val Loss: 0.0608 - Val F1: 0.7167
Epoch 40/150 - Train Loss: 0.0253 - Val Loss: 0.0674 - Val F1: 0.7228
Epoch 43: LR reduced from 0.000210 to 0.000147
Epoch 45/150 - Train Loss: 0.0209 - Val Loss: 0.0759 - Val F1: 0.7405
Epoch 50/150 - Train Loss: 0.0274 - Val Loss: 0.0696 - Val F1: 0.7286
Epoch 51: LR reduced from 0.000147 to 0.000103
Epoch 55/150 - Train Loss: 0.0147 - Val Loss: 0.0738 - Val F1: 0.7506
Early stopping at epoch 55 (best val loss 0.060823)
Completed. Training time: 00:03:54 (234.05s) - Final val loss: 0.0738

```

--- Training Bi-RNN with lr=1e-04 ---

```

Epoch 1/150 - Train Loss: 0.0996 - Val Loss: 0.0870 - Val F1: 0.5344
Epoch 5/150 - Train Loss: 0.0764 - Val Loss: 0.0753 - Val F1: 0.6318
Epoch 10/150 - Train Loss: 0.0687 - Val Loss: 0.0707 - Val F1: 0.6746
Epoch 15/150 - Train Loss: 0.0647 - Val Loss: 0.0666 - Val F1: 0.6612
Epoch 20/150 - Train Loss: 0.0588 - Val Loss: 0.0633 - Val F1: 0.6800

```

Epoch 25/150 - Train Loss: 0.0543 - Val Loss: 0.0738 - Val F1: 0.7107
 Epoch 30/150 - Train Loss: 0.0524 - Val Loss: 0.0548 - Val F1: 0.7509
 Epoch 35/150 - Train Loss: 0.0483 - Val Loss: 0.0613 - Val F1: 0.7450
 Epoch 40/150 - Train Loss: 0.0456 - Val Loss: 0.0684 - Val F1: 0.6733
 Epoch 45/150 - Train Loss: 0.0422 - Val Loss: 0.0530 - Val F1: 0.6961
 Epoch 50/150 - Train Loss: 0.0389 - Val Loss: 0.0465 - Val F1: 0.7537
 Epoch 55/150 - Train Loss: 0.0374 - Val Loss: 0.0532 - Val F1: 0.7440
 Epoch 56: LR reduced from 0.000100 to 0.000070
 Epoch 60/150 - Train Loss: 0.0339 - Val Loss: 0.0486 - Val F1: 0.7806
 Epoch 65/150 - Train Loss: 0.0312 - Val Loss: 0.0463 - Val F1: 0.7473
 Epoch 70/150 - Train Loss: 0.0277 - Val Loss: 0.0452 - Val F1: 0.7378
 Epoch 75/150 - Train Loss: 0.0269 - Val Loss: 0.0488 - Val F1: 0.7701
 Epoch 80/150 - Train Loss: 0.0264 - Val Loss: 0.0496 - Val F1: 0.7568
 Epoch 85/150 - Train Loss: 0.0293 - Val Loss: 0.0493 - Val F1: 0.7691
 Epoch 86: LR reduced from 0.000070 to 0.000049
 Epoch 90/150 - Train Loss: 0.0246 - Val Loss: 0.0538 - Val F1: 0.7850
 Epoch 95/150 - Train Loss: 0.0218 - Val Loss: 0.0478 - Val F1: 0.7717
 Epoch 100/150 - Train Loss: 0.0197 - Val Loss: 0.0442 - Val F1: 0.7915
 Epoch 105/150 - Train Loss: 0.0190 - Val Loss: 0.0464 - Val F1: 0.7687
 Epoch 108: LR reduced from 0.000049 to 0.000034
 Epoch 110/150 - Train Loss: 0.0187 - Val Loss: 0.0454 - Val F1: 0.7809
 Epoch 115/150 - Train Loss: 0.0184 - Val Loss: 0.0472 - Val F1: 0.7750
 Epoch 120/150 - Train Loss: 0.0162 - Val Loss: 0.0508 - Val F1: 0.7917
 Epoch 124: LR reduced from 0.000034 to 0.000024
 Epoch 125/150 - Train Loss: 0.0158 - Val Loss: 0.0455 - Val F1: 0.7799
 Epoch 130/150 - Train Loss: 0.0144 - Val Loss: 0.0472 - Val F1: 0.7806
 Epoch 132: LR reduced from 0.000024 to 0.000017
 Epoch 135/150 - Train Loss: 0.0152 - Val Loss: 0.0512 - Val F1: 0.7821
 Early stopping at epoch 136 (best val loss 0.042049)
 Completed. Training time: 00:09:37 (577.89s) - Final val loss: 0.0500

--- Training Bi-RNN with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1013 - Val Loss: 0.0847 - Val F1: 0.5627
 Epoch 5/150 - Train Loss: 0.0775 - Val Loss: 0.0736 - Val F1: 0.6493
 Epoch 10/150 - Train Loss: 0.0693 - Val Loss: 0.0699 - Val F1: 0.6437
 Epoch 15/150 - Train Loss: 0.0681 - Val Loss: 0.0730 - Val F1: 0.6068
 Epoch 20/150 - Train Loss: 0.0628 - Val Loss: 0.0755 - Val F1: 0.6146
 Epoch 25/150 - Train Loss: 0.0590 - Val Loss: 0.0660 - Val F1: 0.6655
 Epoch 30/150 - Train Loss: 0.0549 - Val Loss: 0.0659 - Val F1: 0.6752
 Epoch 35/150 - Train Loss: 0.0499 - Val Loss: 0.0622 - Val F1: 0.6652
 Epoch 40/150 - Train Loss: 0.0485 - Val Loss: 0.0625 - Val F1: 0.6758
 Epoch 45/150 - Train Loss: 0.0440 - Val Loss: 0.0560 - Val F1: 0.7064
 Epoch 50/150 - Train Loss: 0.0420 - Val Loss: 0.0569 - Val F1: 0.7164
 Epoch 55/150 - Train Loss: 0.0369 - Val Loss: 0.0533 - Val F1: 0.6851
 Epoch 60/150 - Train Loss: 0.0343 - Val Loss: 0.0572 - Val F1: 0.7316
 Epoch 65/150 - Train Loss: 0.0333 - Val Loss: 0.0604 - Val F1: 0.7240
 Epoch 70/150 - Train Loss: 0.0301 - Val Loss: 0.0558 - Val F1: 0.7360
 Epoch 75/150 - Train Loss: 0.0272 - Val Loss: 0.0501 - Val F1: 0.7063

Epoch 80/150 - Train Loss: 0.0255 - Val Loss: 0.0483 - Val F1: 0.7446
 Epoch 85/150 - Train Loss: 0.0216 - Val Loss: 0.0509 - Val F1: 0.7813
 Epoch 90/150 - Train Loss: 0.0256 - Val Loss: 0.0566 - Val F1: 0.7624
 Epoch 95/150 - Train Loss: 0.0211 - Val Loss: 0.0520 - Val F1: 0.7714
 Epoch 100/150 - Train Loss: 0.0208 - Val Loss: 0.0515 - Val F1: 0.8063
 Epoch 102: LR reduced from 0.000080 to 0.000056
 Epoch 105/150 - Train Loss: 0.0171 - Val Loss: 0.0456 - Val F1: 0.7644
 Epoch 110/150 - Train Loss: 0.0195 - Val Loss: 0.0455 - Val F1: 0.7592
 Epoch 112: LR reduced from 0.000056 to 0.000039
 Epoch 115/150 - Train Loss: 0.0137 - Val Loss: 0.0446 - Val F1: 0.8031
 Epoch 120: LR reduced from 0.000039 to 0.000027
 Epoch 120/150 - Train Loss: 0.0134 - Val Loss: 0.0502 - Val F1: 0.8080
 Early stopping at epoch 124 (best val loss 0.040609)
 Completed. Training time: 00:08:42 (522.64s) - Final val loss: 0.0512

--- Training Bi-RNN with lr=5e-05 ---

Epoch 1/150 - Train Loss: 0.1030 - Val Loss: 0.0902 - Val F1: 0.4905
 Epoch 5/150 - Train Loss: 0.0788 - Val Loss: 0.0761 - Val F1: 0.6379
 Epoch 10/150 - Train Loss: 0.0706 - Val Loss: 0.0725 - Val F1: 0.6737
 Epoch 15/150 - Train Loss: 0.0655 - Val Loss: 0.0705 - Val F1: 0.6682
 Epoch 20/150 - Train Loss: 0.0637 - Val Loss: 0.0705 - Val F1: 0.6451
 Epoch 25: LR reduced from 0.000050 to 0.000035
 Epoch 25/150 - Train Loss: 0.0618 - Val Loss: 0.0692 - Val F1: 0.6441
 Epoch 30/150 - Train Loss: 0.0572 - Val Loss: 0.0649 - Val F1: 0.6668
 Epoch 35/150 - Train Loss: 0.0575 - Val Loss: 0.0671 - Val F1: 0.6898
 Epoch 40/150 - Train Loss: 0.0547 - Val Loss: 0.0628 - Val F1: 0.6711
 Epoch 45: LR reduced from 0.000035 to 0.000024
 Epoch 45/150 - Train Loss: 0.0526 - Val Loss: 0.0665 - Val F1: 0.6819
 Epoch 50/150 - Train Loss: 0.0509 - Val Loss: 0.0615 - Val F1: 0.6877
 Epoch 55/150 - Train Loss: 0.0493 - Val Loss: 0.0651 - Val F1: 0.6544
 Epoch 60/150 - Train Loss: 0.0489 - Val Loss: 0.0595 - Val F1: 0.6995
 Epoch 64: LR reduced from 0.000024 to 0.000017
 Epoch 65/150 - Train Loss: 0.0461 - Val Loss: 0.0586 - Val F1: 0.6775
 Epoch 70/150 - Train Loss: 0.0455 - Val Loss: 0.0599 - Val F1: 0.6672
 Epoch 72: LR reduced from 0.000017 to 0.000012
 Epoch 75/150 - Train Loss: 0.0455 - Val Loss: 0.0589 - Val F1: 0.6892
 Early stopping at epoch 76 (best val loss 0.058548)
 Completed. Training time: 00:05:17 (317.73s) - Final val loss: 0.0610

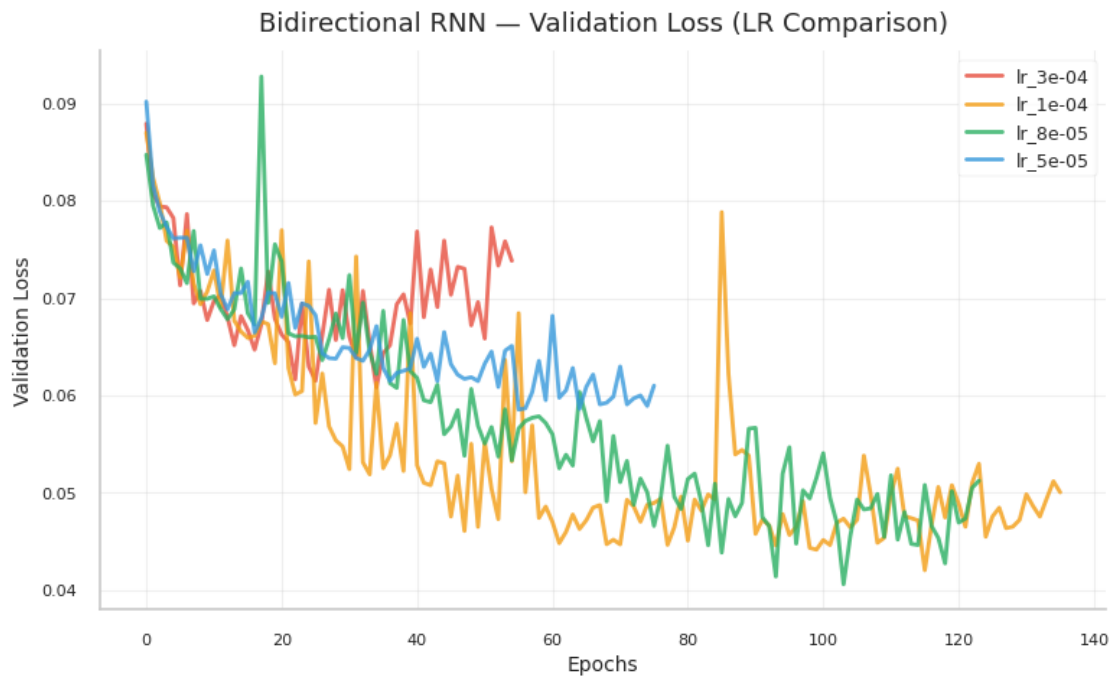
[]: # --- Plot loss curves ---

```
# Plot comparison
plot_lr_comparison(histories_bi, "Bidirectional RNN", save_dir)

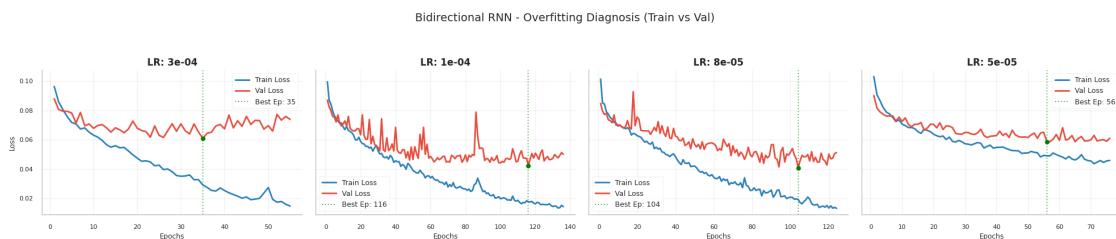
#Plot Train vs Validation
plot_train_val_grid(histories_bi, "Bidirectional RNN", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images

/task3_plots/bidirectional_rnn_lr_comparison.png

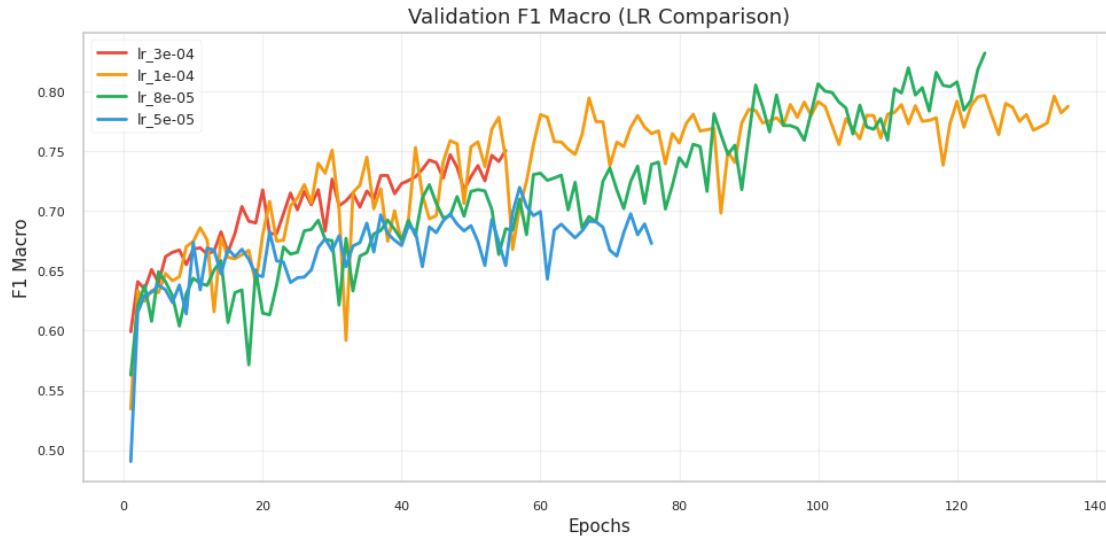


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/bidirectional_rnn_diagnosis.png



```
[ ]: # --- Plot Validation F1 Macro for Bidirectional RNN ---  
plot_training_metrics(histories_seq, save_dir=save_dir,   
    plot_name="bidirectional_rnn_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/comparison_f1_macro.png



```
[ ]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL Bi-RNN learning rates")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]

    print(f"\n{'='*60}")
    print(f"Bi-RNN with lr={lr:.0e}")
    print(f"{'='*60}")

    # Validation evaluation
    y_val_true, y_val_pred = predict_loader(model, val_loader)
    print(f'\nValidation report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_val_true, y_val_pred, digits=4,
    ↪zero_division=0))

    # Test evaluation
    y_test_true, y_test_pred = predict_loader(model, test_loader)
    print(f'Test report (Bi-RNN - lr={lr:.0e}):')
    print(classification_report(y_test_true, y_test_pred, digits=4,
    ↪zero_division=0))

    # Find and highlight best model
    best_lr_bi = min(histories_bi.items(), key=lambda x: min(x[1]['val_loss']))
    print(f"\n{'='*60}")
```

```

print(f" Best Bi-RNN - LR: {best_lr_bi[0]:.0e} (Min Val Loss:␣
↪{min(best_lr_bi[1]['val_loss']):.4f})")
print(f"{' '*60}")

```

```

=====
Evaluation for ALL Bi-RNN learning rates
=====

```

```

=====
Bi-RNN with lr=3e-04
=====

```

Validation report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.3695	0.6148	0.4615	122
1	0.9847	0.9593	0.9718	3143
accuracy			0.9464	3265
macro avg	0.6771	0.7870	0.7167	3265
weighted avg	0.9617	0.9464	0.9527	3265

Test report (Bi-RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.3702	0.5926	0.4557	243
1	0.9838	0.9609	0.9722	6262
accuracy			0.9471	6505
macro avg	0.6770	0.7767	0.7140	6505
weighted avg	0.9609	0.9471	0.9529	6505

```

=====
Bi-RNN with lr=1e-04
=====

```

Validation report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.4780	0.7131	0.5724	122
1	0.9886	0.9698	0.9791	3143
accuracy			0.9602	3265
macro avg	0.7333	0.8414	0.7757	3265
weighted avg	0.9696	0.9602	0.9639	3265

Test report (Bi-RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.4382	0.6708	0.5301	243
1	0.9870	0.9666	0.9767	6262
accuracy			0.9556	6505
macro avg	0.7126	0.8187	0.7534	6505
weighted avg	0.9665	0.9556	0.9600	6505

=====

Bi-RNN with lr=8e-05

=====

Validation report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4945	0.7377	0.5921	122
1	0.9896	0.9707	0.9801	3143
accuracy			0.9620	3265
macro avg	0.7421	0.8542	0.7861	3265
weighted avg	0.9711	0.9620	0.9656	3265

Test report (Bi-RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4392	0.6831	0.5346	243
1	0.9874	0.9661	0.9767	6262
accuracy			0.9556	6505
macro avg	0.7133	0.8246	0.7556	6505
weighted avg	0.9670	0.9556	0.9602	6505

=====

Bi-RNN with lr=5e-05

=====

Validation report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.3304	0.6148	0.4298	122
1	0.9845	0.9516	0.9678	3143
accuracy			0.9391	3265
macro avg	0.6575	0.7832	0.6988	3265

weighted avg 0.9601 0.9391 0.9477 3265

Test report (Bi-RNN - lr=5e-05):

	precision	recall	f1-score	support
0	0.2913	0.5802	0.3879	243
1	0.9831	0.9452	0.9638	6262
accuracy			0.9316	6505
macro avg	0.6372	0.7627	0.6758	6505
weighted avg	0.9572	0.9316	0.9423	6505

=====
Best Bi-RNN - LR: 8e-05 (Min Val Loss: 0.0406)
=====

```
[ ]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - VALIDATION")
print("=" * 80)

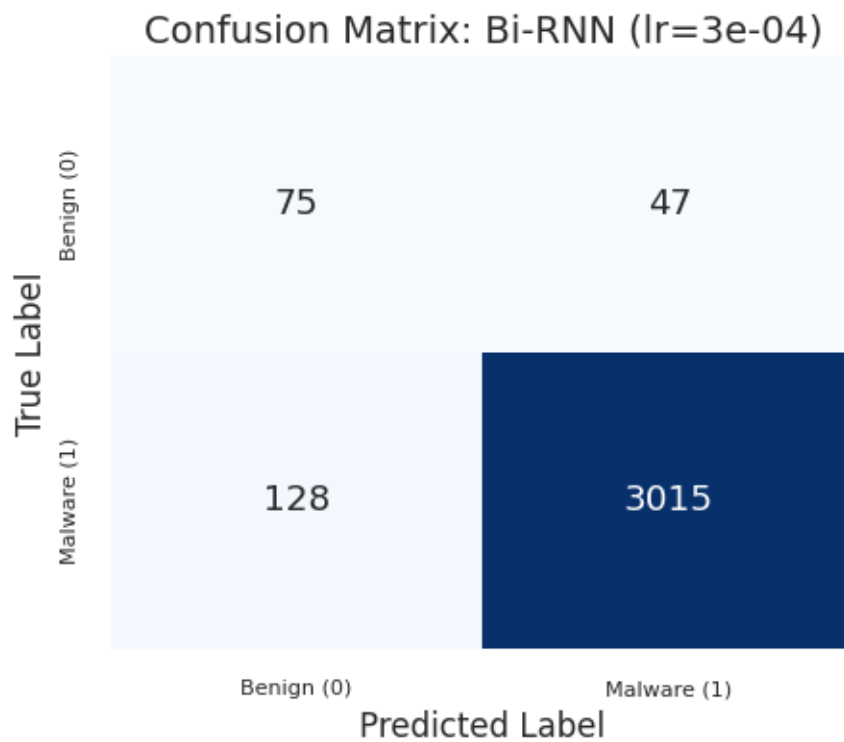
for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"Bi-RNN (lr={lr:.
↪0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL Bi-RNN learning rates - VALIDATION
=====

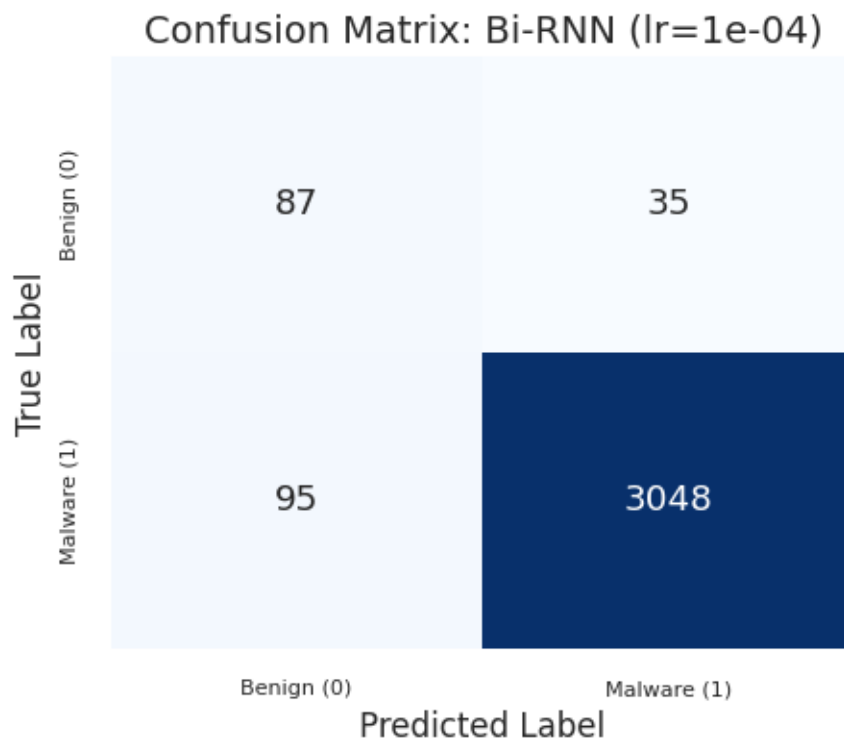
Generating confusion matrix for Bi-RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn_(lr=3e_04).png



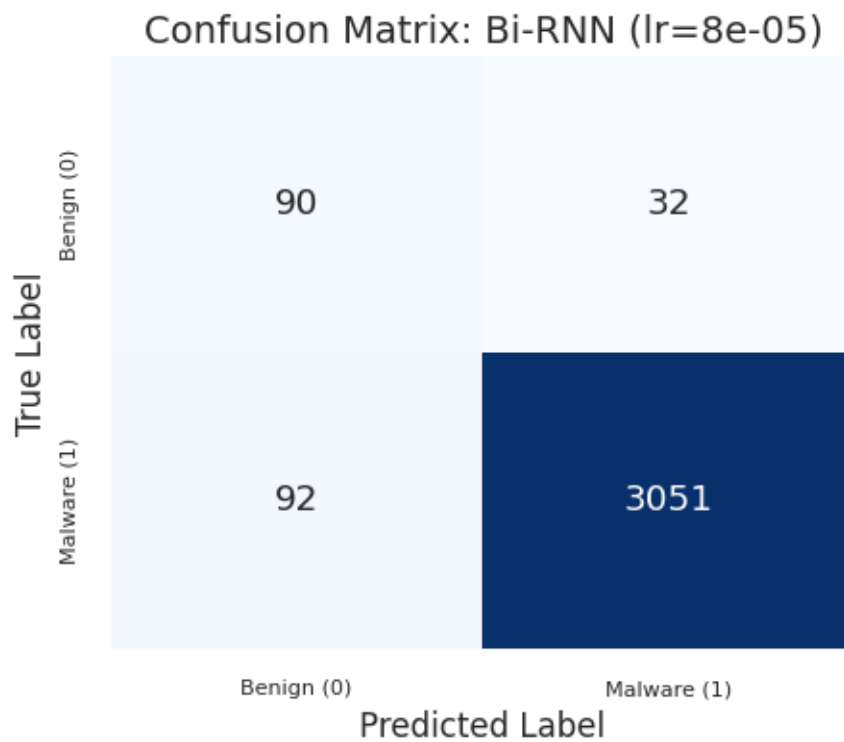
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn(lr=1e_04).png



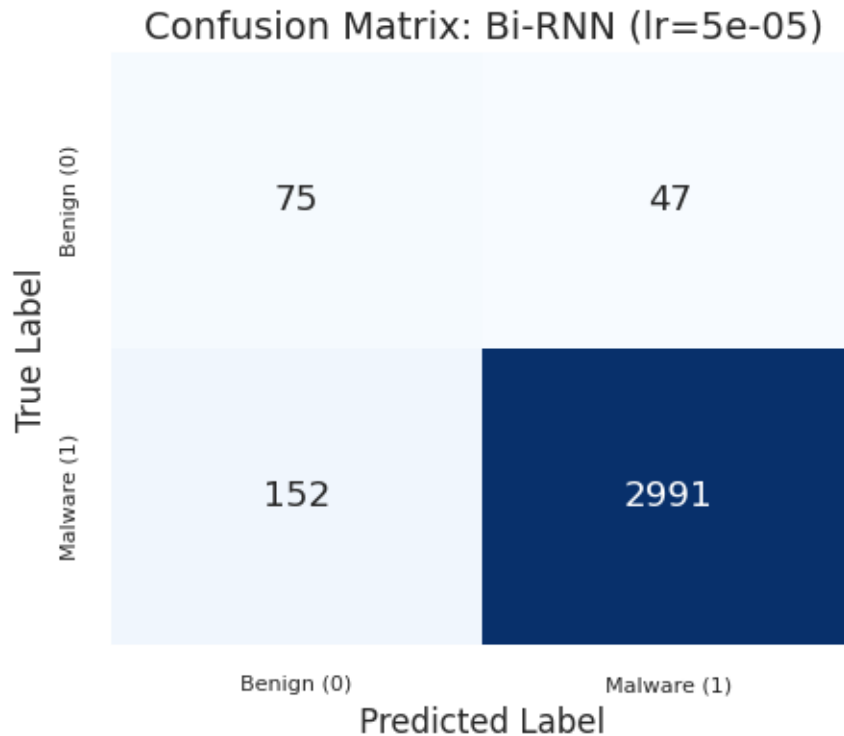
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



Generating confusion matrix for Bi-RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=5e_05).png



```
[ ]: # --- Plot confusion matrices for ALL learning rates - TEST ---

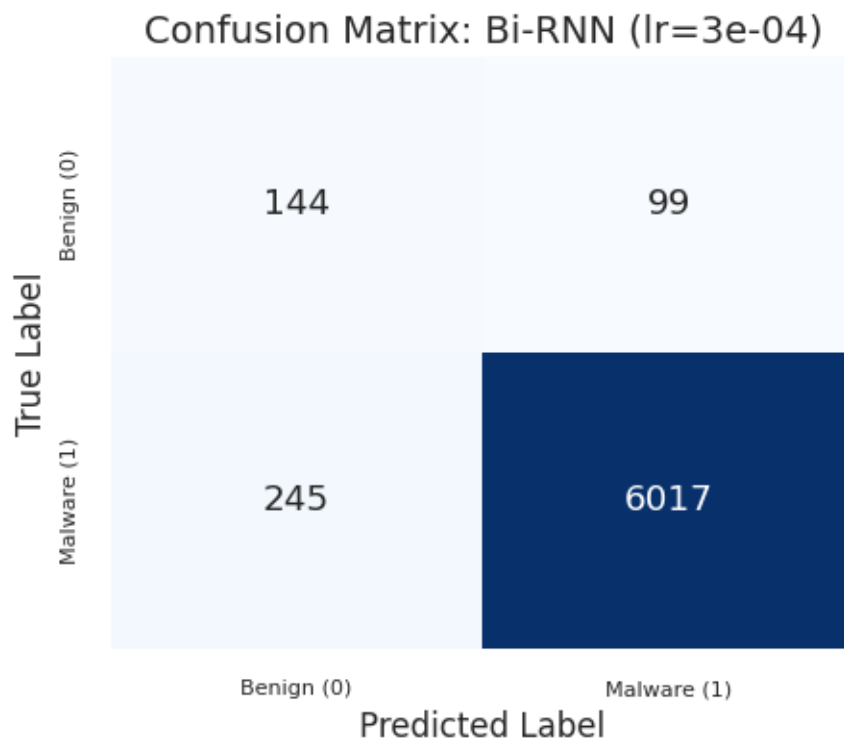
print("\n" + "=" * 80)
print("Confusion matrices for ALL Bi-RNN learning rates - TEST")
print("=" * 80)

for lr in learning_rates:
    model = models_bi[lr]
    print(f"\nGenerating confusion matrix for Bi-RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"Bi-RNN (lr={lr:.0e})", save_dir=save_dir)
```

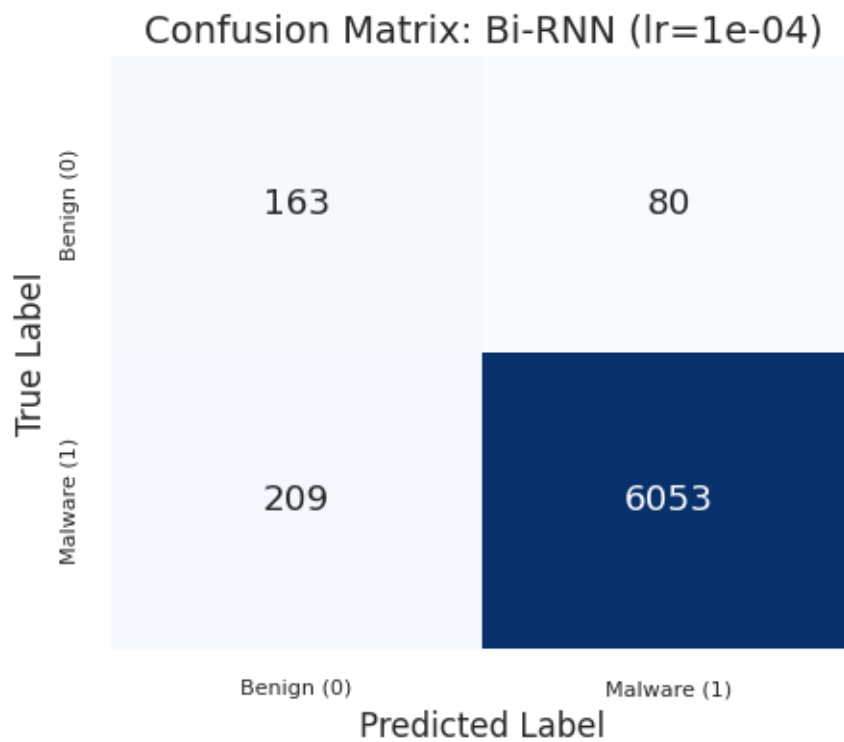
```
=====
Confusion matrices for ALL Bi-RNN learning rates - TEST
=====

Generating confusion matrix for Bi-RNN with lr=3e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_bi_rnn(lr=3e_04).png
```

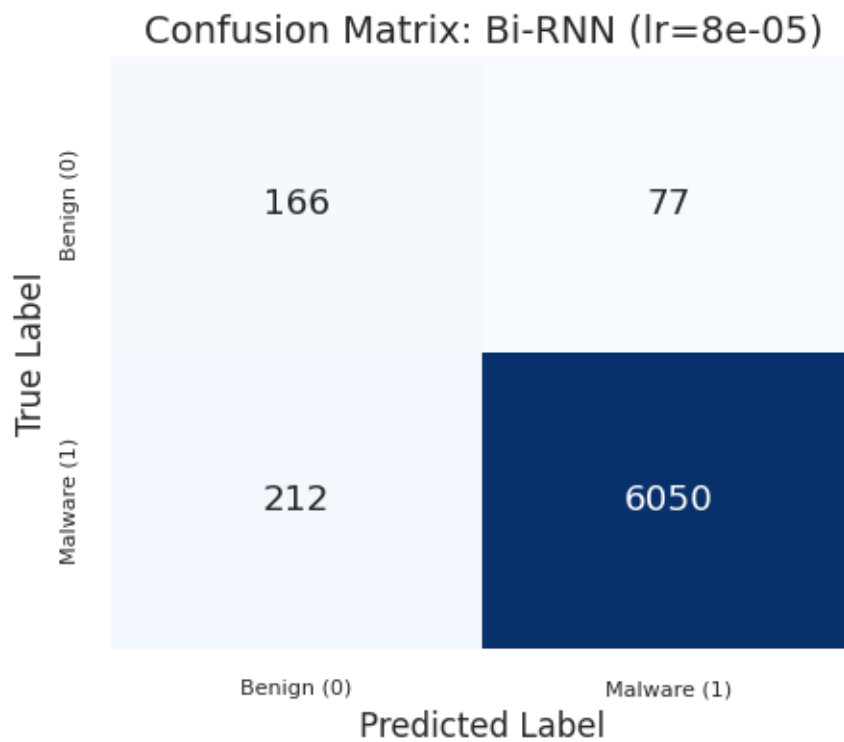
Generating confusion matrix for Bi-RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn(lr=1e_04).png



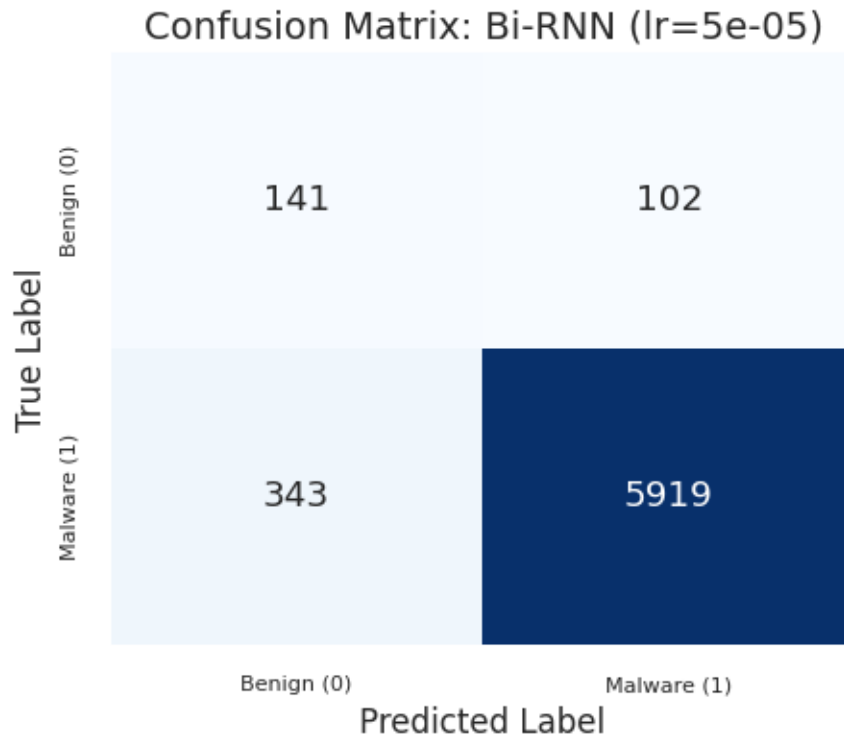
Generating confusion matrix for Bi-RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=8e_05).png



Generating confusion matrix for Bi-RNN with lr=5e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_bi_rnn_(lr=5e_05).png



```
[ ]: # --- Variation 3: LSTM (bidirectional) ---

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=128, num_layers=2,
        ↪ num_classes=1, bidirectional=True, dropout=0.3, padding=0):
        super(LSTMClassifier, self).__init__()

        # Embedding layer: Maps API call indices to dense vectors. padding_idx
        ↪ is set.
        self.embedding = nn.Embedding(vocab_size, embedding_dim=embed_dim,
        ↪ padding_idx=padding)

        # LSTM layer: A more complex RNN unit that helps prevent vanishing
        ↪ gradients.
        # batch_first=True expects input shape (batch_size, seq_len, features).
        self.lstm = nn.LSTM(input_size=embed_dim, hidden_size=hidden_size,
        ↪ num_layers=num_layers,
                               batch_first=True, bidirectional=bidirectional)

        self.bidirectional = bidirectional
        fc_in = hidden_size * (2 if bidirectional else 1)
```

```

self.fc = nn.Linear(fc_in, num_classes)
self.dropout = nn.Dropout(dropout)

def forward(self, x, lengths):
    # Embed the input indices
    emb = self.embedding(x) # (batch_size, max_seq_len, embed_dim)
    emb = self.dropout(emb)

    # Pack the padded sequence to efficiently process variable lengths
    packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True,
    ↪enforce_sorted=True)

    # LSTM returns packed_output, (final_hidden_state, final_cell_state)
    packed_out, (hn, cn) = self.lstm(packed)

    # hn shape: (num_layers * num_directions, batch_size, hidden_size)

    # Get the final hidden state from the last layer
    if self.bidirectional:
        # Concatenate the final forward state (hn[-2]) and backward state
    ↪(hn[-1])
        h_last = torch.cat([hn[-2], hn[-1]], dim=1)
    else:
        # Use the final hidden state of the last layer (hn[-1])
        h_last = hn[-1]

    # Pass the final hidden state through the classifier
    out = self.fc(h_last)
    return out

```

```

[ ]: # --- Variation 3: LSTM (bidirectional) ---

print("\n" + "=" * 80)
print("Training LSTM with different learning rates")
print("=" * 80)

# Model parameters
lstm_params = {
    'vocab_size': vocab_size_rnn,
    'embed_dim': 32,
    'hidden_size': 128,
    'num_layers': 2,
    'num_classes': 1,
    'bidirectional': True,
    'dropout': 0.4
}

```

```

# Initialize early stopping parameters
min_delta = 0.00001
patience = 30

# Scheduler parameters
scheduler_params_lstm = {
    'mode': 'min',
    'factor': 0.5,
    'patience': 6,
    'min_lr': 1e-7
}

# Test slightly different LRs for LSTM (typically handles higher LRs better)
learning_rates_lstm = [5e-04, 3e-04, 1e-04, 8e-05]

# Store histories and models for each LR
histories_lstm = {}
models_lstm = {}
elapsed_lstm = {}

for lr in learning_rates_lstm:
    print(f"\n--- Training LSTM with lr={lr:.0e} ---")
    model, history, elapsed = train_with_lr(
        model_class=LSTMClassifier,
        model_params=lstm_params,
        initial_lr=lr,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight),
        num_epochs=num_epochs,
        min_delta=min_delta,
        patience=patience,
        scheduler_params=scheduler_params_lstm,
        device=device
    )
    histories_lstm[lr] = history
    models_lstm[lr] = model
    elapsed_lstm[lr] = elapsed
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f"Completed. Training time: {elapsed_str} ({elapsed:.2f}s) - Final_
    ↪val loss: {history['val_loss'][-1]:.4f}")

```

```

=====
Training LSTM with different learning rates
=====

```

```

--- Training LSTM with lr=5e-04 ---

```

Epoch 1/150 - Train Loss: 0.0922 - Val Loss: 0.0831 - Val F1: 0.5776
 Epoch 5/150 - Train Loss: 0.0652 - Val Loss: 0.0661 - Val F1: 0.6944
 Epoch 10/150 - Train Loss: 0.0498 - Val Loss: 0.0596 - Val F1: 0.7066
 Epoch 15/150 - Train Loss: 0.0401 - Val Loss: 0.0522 - Val F1: 0.7389
 Epoch 20/150 - Train Loss: 0.0361 - Val Loss: 0.0414 - Val F1: 0.7698
 Epoch 25/150 - Train Loss: 0.0246 - Val Loss: 0.0389 - Val F1: 0.7954
 Epoch 28: LR reduced from 0.000500 to 0.000250
 Epoch 30/150 - Train Loss: 0.0161 - Val Loss: 0.0388 - Val F1: 0.8256
 Epoch 35: LR reduced from 0.000250 to 0.000125
 Epoch 35/150 - Train Loss: 0.0139 - Val Loss: 0.0449 - Val F1: 0.8350
 Epoch 40/150 - Train Loss: 0.0113 - Val Loss: 0.0464 - Val F1: 0.8339
 Epoch 42: LR reduced from 0.000125 to 0.000063
 Epoch 45/150 - Train Loss: 0.0092 - Val Loss: 0.0474 - Val F1: 0.8444
 Epoch 49: LR reduced from 0.000063 to 0.000031
 Epoch 50/150 - Train Loss: 0.0083 - Val Loss: 0.0463 - Val F1: 0.8472
 Early stopping at epoch 51 (best val loss 0.037080)
 Completed. Training time: 00:03:59 (239.64s) - Final val loss: 0.0472

--- Training LSTM with lr=3e-04 ---

Epoch 1/150 - Train Loss: 0.0947 - Val Loss: 0.0824 - Val F1: 0.4905
 Epoch 5/150 - Train Loss: 0.0668 - Val Loss: 0.0671 - Val F1: 0.6488
 Epoch 10/150 - Train Loss: 0.0513 - Val Loss: 0.0561 - Val F1: 0.6824
 Epoch 15/150 - Train Loss: 0.0456 - Val Loss: 0.0494 - Val F1: 0.7386
 Epoch 20/150 - Train Loss: 0.0399 - Val Loss: 0.0429 - Val F1: 0.7727
 Epoch 25/150 - Train Loss: 0.0291 - Val Loss: 0.0313 - Val F1: 0.8316
 Epoch 30/150 - Train Loss: 0.0271 - Val Loss: 0.0325 - Val F1: 0.8427
 Epoch 35/150 - Train Loss: 0.0248 - Val Loss: 0.0288 - Val F1: 0.8346
 Epoch 40/150 - Train Loss: 0.0199 - Val Loss: 0.0305 - Val F1: 0.8315
 Epoch 42: LR reduced from 0.000300 to 0.000150
 Epoch 45/150 - Train Loss: 0.0145 - Val Loss: 0.0316 - Val F1: 0.8465
 Epoch 49: LR reduced from 0.000150 to 0.000075
 Epoch 50/150 - Train Loss: 0.0119 - Val Loss: 0.0320 - Val F1: 0.8535
 Epoch 55/150 - Train Loss: 0.0107 - Val Loss: 0.0329 - Val F1: 0.8457
 Epoch 56: LR reduced from 0.000075 to 0.000037
 Epoch 60/150 - Train Loss: 0.0090 - Val Loss: 0.0337 - Val F1: 0.8463
 Epoch 63: LR reduced from 0.000037 to 0.000019
 Epoch 65/150 - Train Loss: 0.0086 - Val Loss: 0.0345 - Val F1: 0.8504
 Early stopping at epoch 65 (best val loss 0.028766)
 Completed. Training time: 00:05:03 (303.85s) - Final val loss: 0.0345

--- Training LSTM with lr=1e-04 ---

Epoch 1/150 - Train Loss: 0.1081 - Val Loss: 0.0838 - Val F1: 0.4904
 Epoch 5/150 - Train Loss: 0.0738 - Val Loss: 0.0696 - Val F1: 0.6335
 Epoch 10/150 - Train Loss: 0.0643 - Val Loss: 0.0615 - Val F1: 0.6701
 Epoch 15/150 - Train Loss: 0.0636 - Val Loss: 0.0680 - Val F1: 0.6562
 Epoch 20/150 - Train Loss: 0.0549 - Val Loss: 0.0552 - Val F1: 0.6949
 Epoch 25/150 - Train Loss: 0.0478 - Val Loss: 0.0543 - Val F1: 0.6812
 Epoch 30/150 - Train Loss: 0.0451 - Val Loss: 0.0487 - Val F1: 0.7142

Epoch 35/150 - Train Loss: 0.0410 - Val Loss: 0.0438 - Val F1: 0.7343
 Epoch 40/150 - Train Loss: 0.0397 - Val Loss: 0.0464 - Val F1: 0.7332
 Epoch 42: LR reduced from 0.000100 to 0.000050
 Epoch 45/150 - Train Loss: 0.0329 - Val Loss: 0.0411 - Val F1: 0.7852
 Epoch 50/150 - Train Loss: 0.0318 - Val Loss: 0.0388 - Val F1: 0.7885
 Epoch 55/150 - Train Loss: 0.0304 - Val Loss: 0.0377 - Val F1: 0.7788
 Epoch 60/150 - Train Loss: 0.0272 - Val Loss: 0.0393 - Val F1: 0.7782
 Epoch 62: LR reduced from 0.000050 to 0.000025
 Epoch 65/150 - Train Loss: 0.0271 - Val Loss: 0.0374 - Val F1: 0.7818
 Epoch 70/150 - Train Loss: 0.0260 - Val Loss: 0.0386 - Val F1: 0.7950
 Epoch 74: LR reduced from 0.000025 to 0.000013
 Epoch 75/150 - Train Loss: 0.0243 - Val Loss: 0.0372 - Val F1: 0.7948
 Epoch 80/150 - Train Loss: 0.0246 - Val Loss: 0.0366 - Val F1: 0.7961
 Epoch 85/150 - Train Loss: 0.0241 - Val Loss: 0.0365 - Val F1: 0.7911
 Epoch 90/150 - Train Loss: 0.0227 - Val Loss: 0.0382 - Val F1: 0.8015
 Epoch 95: LR reduced from 0.000013 to 0.000006
 Epoch 95/150 - Train Loss: 0.0229 - Val Loss: 0.0377 - Val F1: 0.8028
 Epoch 100/150 - Train Loss: 0.0236 - Val Loss: 0.0372 - Val F1: 0.7936
 Epoch 102: LR reduced from 0.000006 to 0.000003
 Epoch 105/150 - Train Loss: 0.0225 - Val Loss: 0.0367 - Val F1: 0.7930
 Epoch 109: LR reduced from 0.000003 to 0.000002
 Epoch 110/150 - Train Loss: 0.0217 - Val Loss: 0.0369 - Val F1: 0.7939
 Epoch 115/150 - Train Loss: 0.0231 - Val Loss: 0.0369 - Val F1: 0.7975
 Epoch 116: LR reduced from 0.000002 to 0.000001
 Early stopping at epoch 118 (best val loss 0.035805)
 Completed. Training time: 00:09:17 (557.55s) - Final val loss: 0.0371

--- Training LSTM with lr=8e-05 ---

Epoch 1/150 - Train Loss: 0.1114 - Val Loss: 0.0911 - Val F1: 0.4905
 Epoch 5/150 - Train Loss: 0.0786 - Val Loss: 0.0776 - Val F1: 0.5701
 Epoch 10/150 - Train Loss: 0.0702 - Val Loss: 0.0703 - Val F1: 0.6392
 Epoch 15/150 - Train Loss: 0.0634 - Val Loss: 0.0647 - Val F1: 0.6889
 Epoch 20/150 - Train Loss: 0.0598 - Val Loss: 0.0667 - Val F1: 0.6322
 Epoch 25/150 - Train Loss: 0.0535 - Val Loss: 0.0564 - Val F1: 0.7148
 Epoch 30/150 - Train Loss: 0.0508 - Val Loss: 0.0556 - Val F1: 0.7143
 Epoch 35/150 - Train Loss: 0.0478 - Val Loss: 0.0557 - Val F1: 0.6600
 Epoch 40/150 - Train Loss: 0.0446 - Val Loss: 0.0508 - Val F1: 0.7056
 Epoch 45/150 - Train Loss: 0.0428 - Val Loss: 0.0506 - Val F1: 0.7324
 Epoch 46: LR reduced from 0.000080 to 0.000040
 Epoch 50/150 - Train Loss: 0.0394 - Val Loss: 0.0472 - Val F1: 0.7478
 Epoch 55/150 - Train Loss: 0.0369 - Val Loss: 0.0472 - Val F1: 0.7606
 Epoch 60/150 - Train Loss: 0.0359 - Val Loss: 0.0466 - Val F1: 0.7319
 Epoch 63: LR reduced from 0.000040 to 0.000020
 Epoch 65/150 - Train Loss: 0.0331 - Val Loss: 0.0458 - Val F1: 0.7659
 Epoch 70/150 - Train Loss: 0.0350 - Val Loss: 0.0454 - Val F1: 0.7746
 Epoch 75: LR reduced from 0.000020 to 0.000010
 Epoch 75/150 - Train Loss: 0.0317 - Val Loss: 0.0479 - Val F1: 0.7743
 Epoch 80/150 - Train Loss: 0.0325 - Val Loss: 0.0464 - Val F1: 0.7691

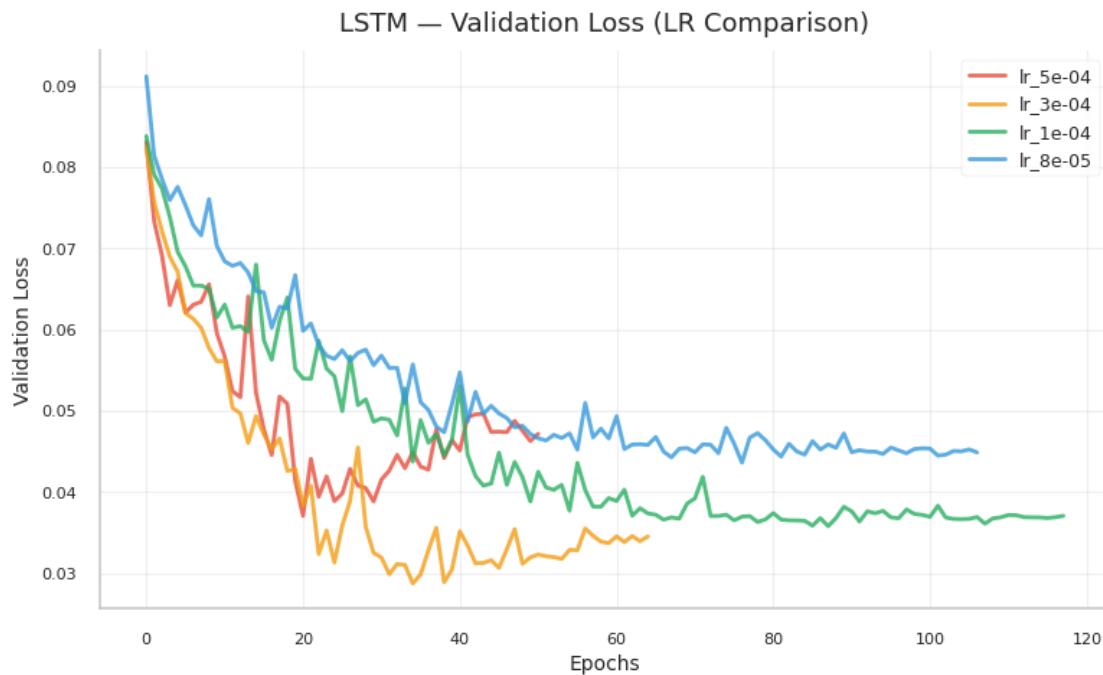
Epoch 84: LR reduced from 0.000010 to 0.000005
Epoch 85/150 - Train Loss: 0.0329 - Val Loss: 0.0446 - Val F1: 0.7629
Epoch 90/150 - Train Loss: 0.0290 - Val Loss: 0.0472 - Val F1: 0.7754
Epoch 91: LR reduced from 0.000005 to 0.000003
Epoch 95/150 - Train Loss: 0.0319 - Val Loss: 0.0447 - Val F1: 0.7664
Epoch 98: LR reduced from 0.000003 to 0.000001
Epoch 100/150 - Train Loss: 0.0309 - Val Loss: 0.0454 - Val F1: 0.7759
Epoch 105: LR reduced from 0.000001 to 0.000001
Epoch 105/150 - Train Loss: 0.0302 - Val Loss: 0.0450 - Val F1: 0.7727
Early stopping at epoch 107 (best val loss 0.043627)
Completed. Training time: 00:08:19 (499.73s) - Final val loss: 0.0449

```
[ ]: # --- Plot loss curves ---

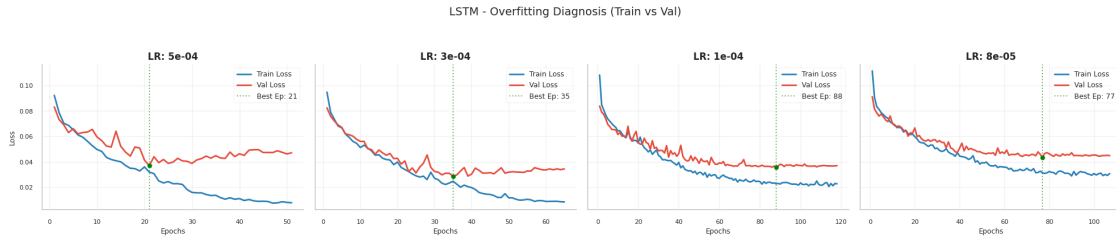
# Plot comparison
plot_lr_comparison(histories_lstm, "LSTM", save_dir)

#Plot Train vs Validation
plot_train_val_grid(histories_lstm, "LSTM", save_dir)
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_lr_comparison.png

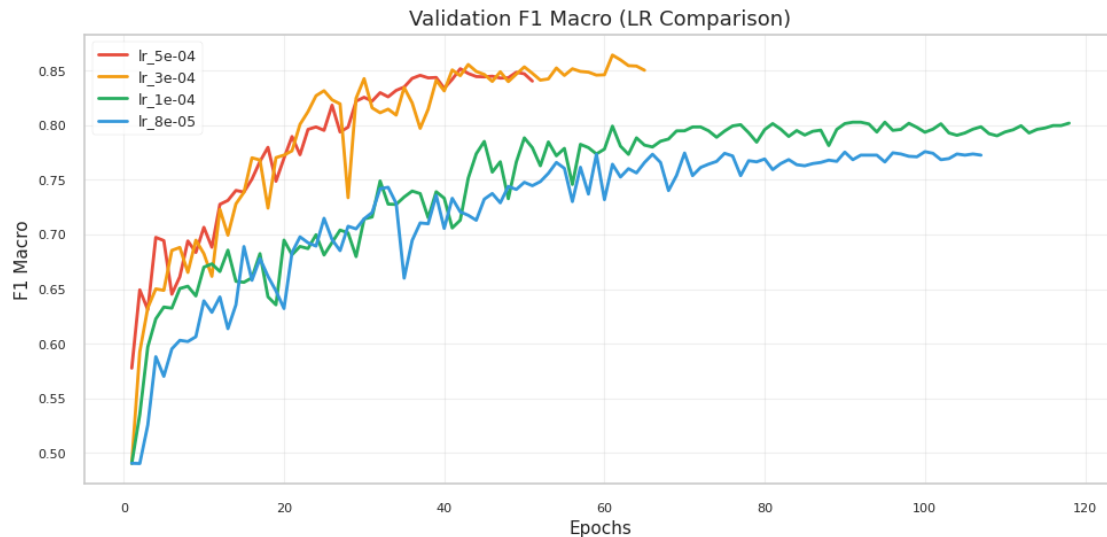


Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/lstm_diagnosis.png



```
[ ]: # --- Plot Validation F1 Macro for LSTM ---
plot_training_metrics(histories_seq, save_dir=save_dir,
↳ plot_name="lstm_f1_comparison")
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/comparison_f1_macro.png



```
[ ]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL LSTM RNN learning rates")
print("=" * 80)

for lr in learning_rates_lstm: # Changed 'learning_rates' to
↳ 'learning_rates_lstm'
    model = models_lstm[lr]

    print(f"\n{'='*60}")
    print(f"LSTM RNN with lr={lr:.0e}")
```

```

print(f"{'='*60}")

# Validation evaluation
y_val_true, y_val_pred = predict_loader(model, val_loader)
print(f'\nValidation report (LSTM RNN - lr={lr:.0e}):')
print(classification_report(y_val_true, y_val_pred, digits=4,
↪zero_division=0))

# Test evaluation
y_test_true, y_test_pred = predict_loader(model, test_loader)
print(f'Test report (LSTM RNN - lr={lr:.0e}):')
print(classification_report(y_test_true, y_test_pred, digits=4,
↪zero_division=0))

# Find and highlight best model
best_lr_lstm = min(histories_lstm.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best LSTM RNN - LR: {best_lr_lstm[0]:.0e} (Min Val Loss:
↪{min(best_lr_lstm[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL LSTM RNN learning rates
=====

```

```

=====
LSTM RNN with lr=5e-04
=====

```

Validation report (LSTM RNN - lr=5e-04):

	precision	recall	f1-score	support
0	0.4848	0.7869	0.6000	122
1	0.9915	0.9675	0.9794	3143
accuracy			0.9608	3265
macro avg	0.7382	0.8772	0.7897	3265
weighted avg	0.9726	0.9608	0.9652	3265

Test report (LSTM RNN - lr=5e-04):

	precision	recall	f1-score	support
0	0.4551	0.6667	0.5409	243
1	0.9868	0.9690	0.9778	6262
accuracy			0.9577	6505

macro avg	0.7209	0.8178	0.7594	6505
weighted avg	0.9670	0.9577	0.9615	6505

=====

LSTM RNN with lr=3e-04

=====

Validation report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.5754	0.8443	0.6844	122
1	0.9938	0.9758	0.9847	3143
accuracy			0.9709	3265
macro avg	0.7846	0.9100	0.8346	3265
weighted avg	0.9782	0.9709	0.9735	3265

Test report (LSTM RNN - lr=3e-04):

	precision	recall	f1-score	support
0	0.5387	0.7449	0.6252	243
1	0.9899	0.9752	0.9825	6262
accuracy			0.9666	6505
macro avg	0.7643	0.8601	0.8039	6505
weighted avg	0.9731	0.9666	0.9692	6505

=====

LSTM RNN with lr=1e-04

=====

Validation report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.4680	0.7787	0.5846	122
1	0.9912	0.9656	0.9782	3143
accuracy			0.9587	3265
macro avg	0.7296	0.8722	0.7814	3265
weighted avg	0.9716	0.9587	0.9635	3265

Test report (LSTM RNN - lr=1e-04):

	precision	recall	f1-score	support
0	0.4737	0.7407	0.5778	243
1	0.9897	0.9681	0.9788	6262

accuracy			0.9596	6505
macro avg	0.7317	0.8544	0.7783	6505
weighted avg	0.9704	0.9596	0.9638	6505

=====

LSTM RNN with lr=8e-05

=====

Validation report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4330	0.6885	0.5316	122
1	0.9876	0.9650	0.9762	3143

accuracy			0.9547	3265
macro avg	0.7103	0.8268	0.7539	3265
weighted avg	0.9669	0.9547	0.9596	3265

Test report (LSTM RNN - lr=8e-05):

	precision	recall	f1-score	support
0	0.4548	0.6831	0.5461	243
1	0.9875	0.9682	0.9777	6262

accuracy			0.9576	6505
macro avg	0.7211	0.8257	0.7619	6505
weighted avg	0.9676	0.9576	0.9616	6505

=====

Best LSTM RNN - LR: 3e-04 (Min Val Loss: 0.0288)

=====

```
[ ]: # --- Plot confusion matrices for ALL learning rates - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - VALIDATION")
print("=" * 80)

for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=val_loader, model_name=f"LSTM RNN (lr={lr:
↵.0e})", save_dir=save_dir)
```

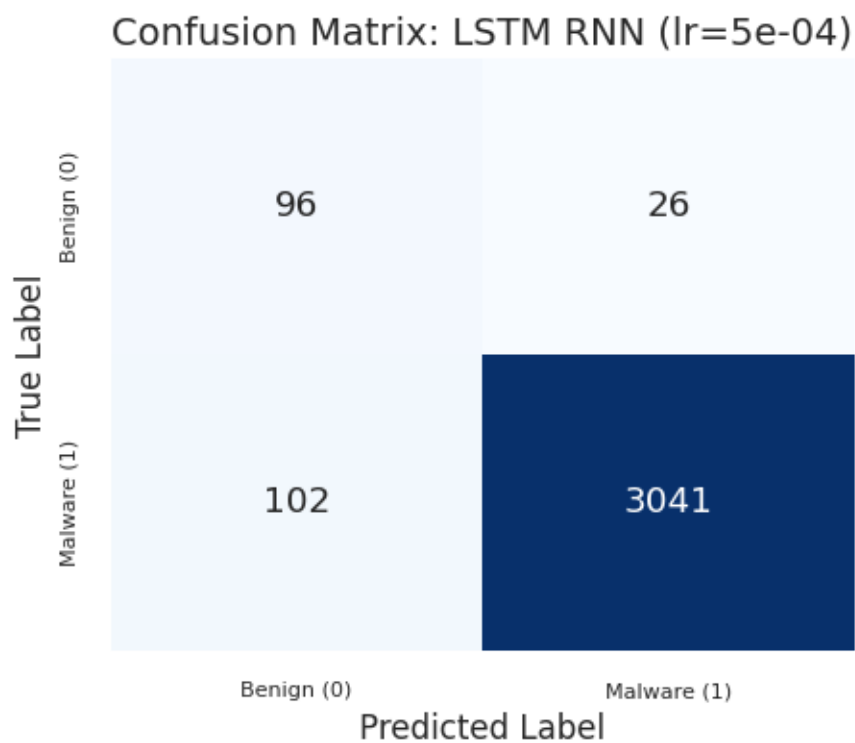
=====

Confusion matrices for ALL LSTM RNN learning rates - VALIDATION

=====

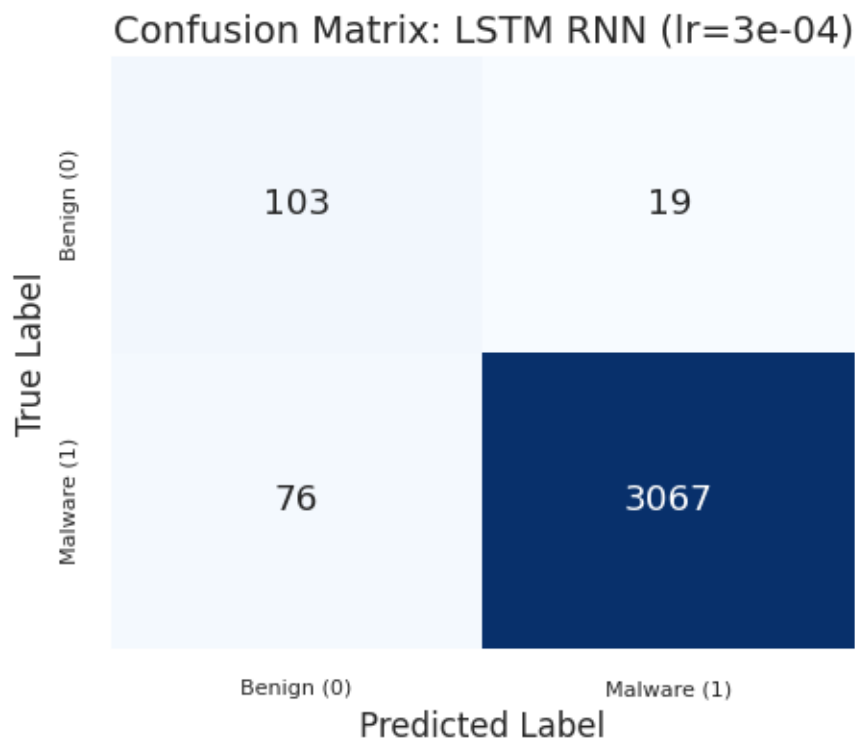
Generating confusion matrix for LSTM RNN with lr=5e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=5e_04).png



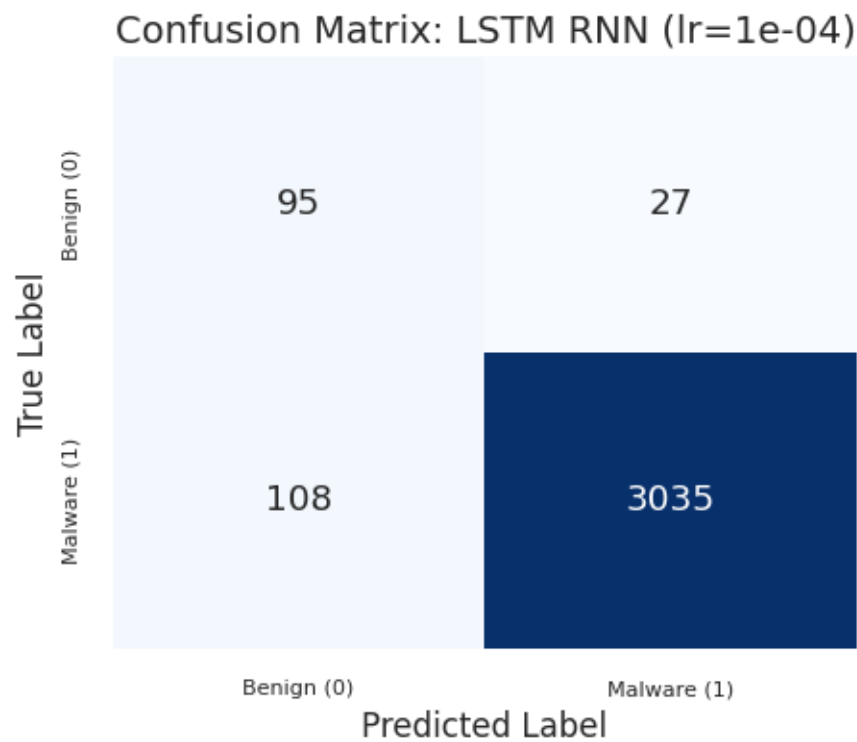
Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=3e_04).png



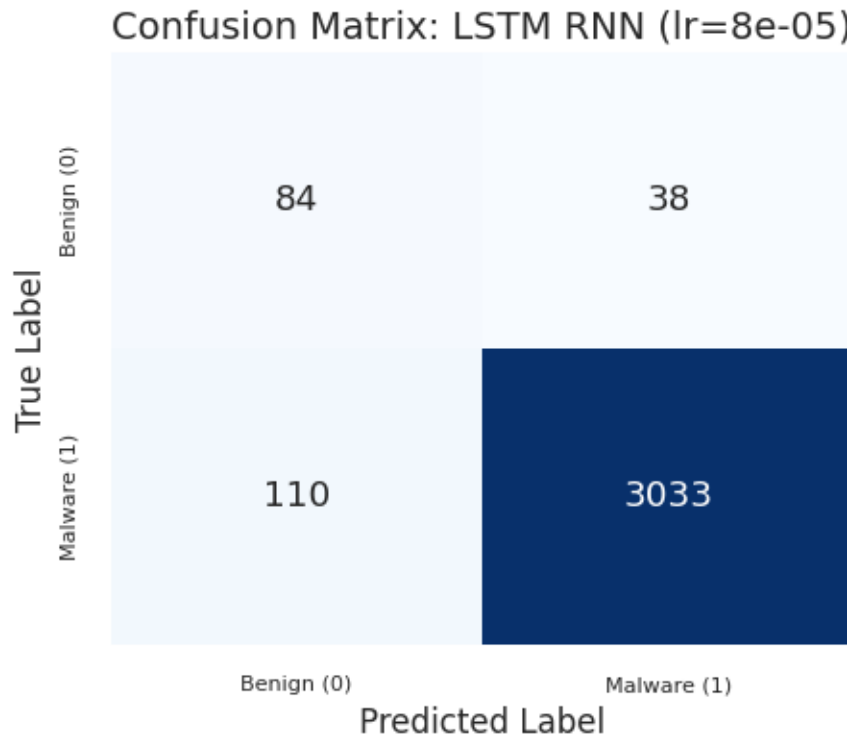
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



```
[ ]: # --- Plot confusion matrices for ALL learning rates - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL LSTM RNN learning rates - TEST")
print("=" * 80)

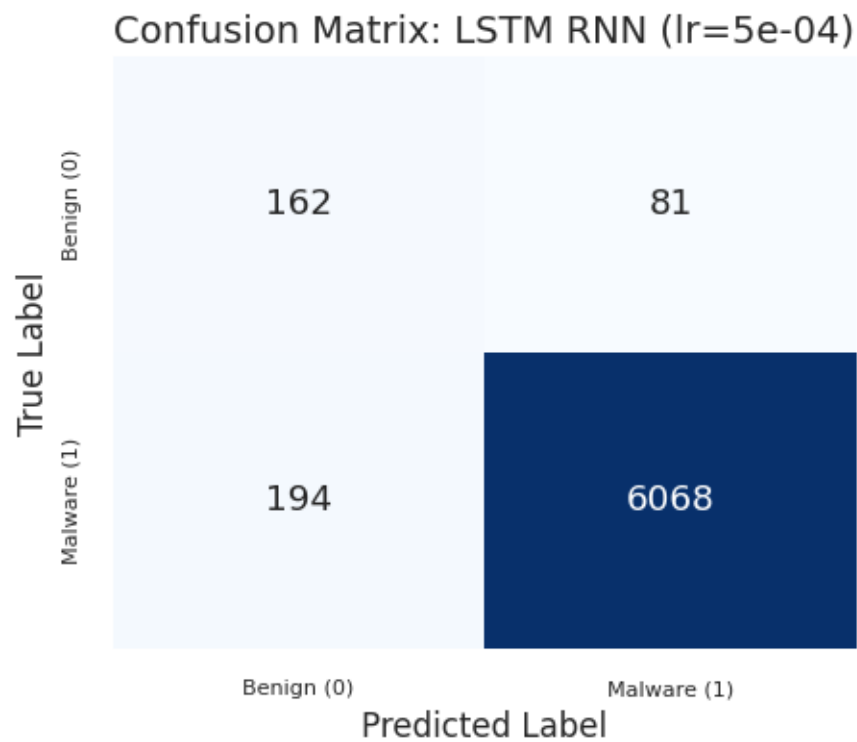
for lr in learning_rates_lstm:
    model = models_lstm[lr]
    print(f"\nGenerating confusion matrix for LSTM RNN with lr={lr:.0e}...")

    plot_cm_task3(model=model, loader=test_loader, model_name=f"LSTM RNN_
↳(lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL LSTM RNN learning rates - TEST
=====
```

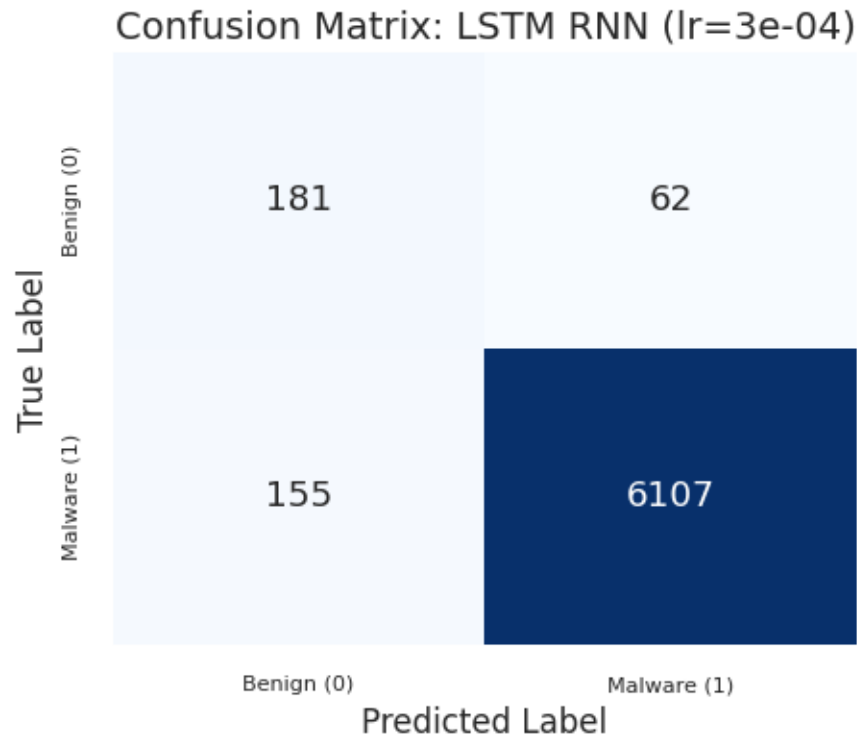
```
Generating confusion matrix for LSTM RNN with lr=5e-04...
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task3_plots/cm_lstm_rnn_(lr=5e_04).png
```



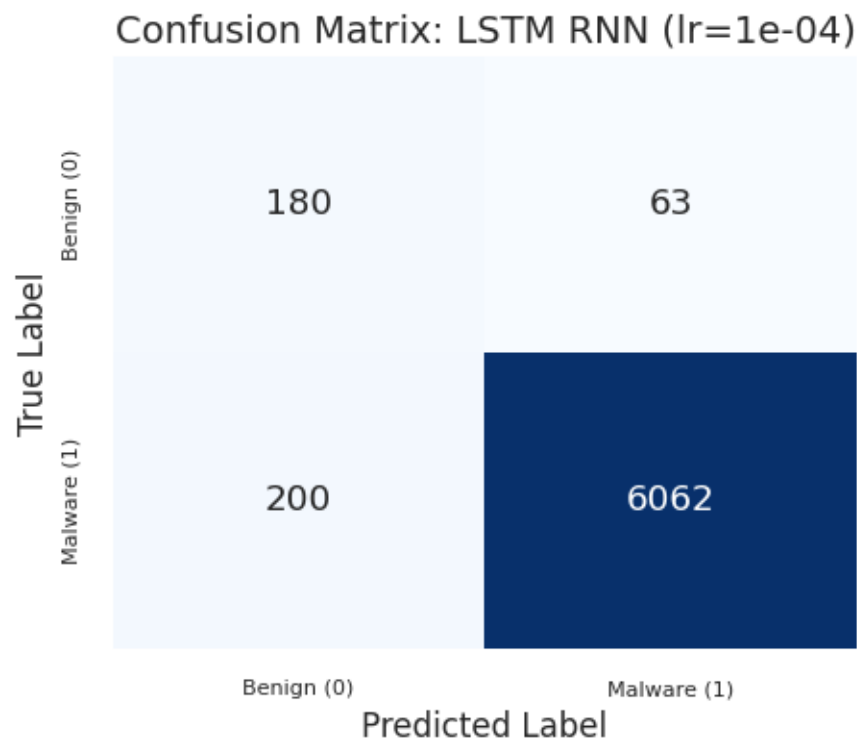
Generating confusion matrix for LSTM RNN with lr=3e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=3e_04).png



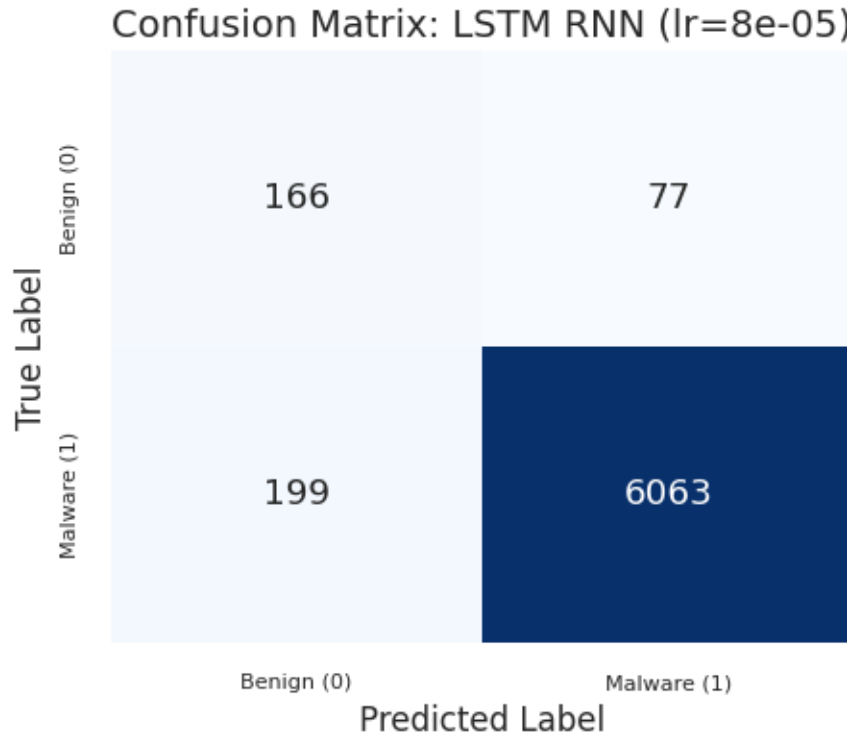
Generating confusion matrix for LSTM RNN with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=1e_04).png



Generating confusion matrix for LSTM RNN with lr=8e-05...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task3_plots/cm_lstm_rnn_(lr=8e_05).png



Q: Is the RNN training as stable as the FFNN's one? No, the RNN training is noticeably **less stable** than the FFNN's. While our FFNN loss curves (Task 2) showed a smooth, monotonic decrease, the RNN variations exhibited more fluctuations and sensitivity to hyperparameters.

1. **Gradient Path Complexity:** Unlike FFNNs, where gradients flow through a fixed number of layers, RNNs must backpropagate through time (BPTT). For our sequences of up to 100 API calls, this creates a much deeper computational graph, increasing the risk of **vanishing and exploding gradients**.
2. **Stability of LSTM:** We observed that the LSTM was the most stable among the recurrent architectures. Thanks to its gating mechanisms (input, forget, and output gates), it effectively mitigated the vanishing gradient problem, allowing it to reach a lower **validation loss** compared to the **Simple RNN**.
3. **Oscillations in Validation Loss:** As seen in the Overfitting Diagnosis plots, the validation curves for RNNs are “spikier.” This suggests that the hidden state dynamics are highly sensitive to small weight updates, making the optimization landscape more complex than the relatively straightforward convex-like surface of a standard FFNN.

Q: How does your model's performance compare to the simple frequency baseline? While the simple frequency **baseline (Logistic Regression)** performed surprisingly well in terms of **total accuracy**, the RNN models—particularly the Bidirectional LSTM—provide a much more robust and “security-aware” classification.

Key Observations:

1. **The Accuracy Trap:** The baseline has a very high accuracy, but it is heavily biased toward the majority class. It fails to identify nearly of benign samples.
2. **Superior Minority Class Detection:** The LSTM dramatically improves the detection of the minority class, achieving a better Benign Recall (more than double the baseline). In a real-world cybersecurity pipeline, this is crucial as it significantly reduces the number of false positives (benign files flagged as malware).
3. **Beyond Frequency:** The baseline only knows how many times an API was called. The fact that the LSTM achieves a higher Macro F1 proves that the order and temporal context of API calls (e.g., opening a file before writing to it) contain discriminative patterns that frequency counts alone cannot capture.
4. **Trade-off:** The baseline remains the best at catching every single malware (Recall 0.99), but at the cost of being almost useless for benign file identification. The LSTM offers a much better balance for a production environment.

```
[ ]: # =====  
# FINAL SUMMARY  
# =====  
  
# Helper function to extract macro avg from the best model  
def get_macro_avg_stats(model, loader):  
    # Predict using your existing helper function  
    y_true, y_pred = predict_loader(model, loader)  
    # Get report as a DICTIONARY, not a string  
    report_dict = classification_report(y_true, y_pred, output_dict=True,  
    ↪ zero_division=0)  
    return report_dict['macro avg']  
  
# --- 1. Prepare Stats for Simple RNN ---  
# Retrieve the actual model object using the best LR key  
best_model_simple_obj = models_simple[best_lr_simple[0]]  
macro_simple = get_macro_avg_stats(best_model_simple_obj, val_loader)  
  
# --- 2. Prepare Stats for Bi-RNN ---  
best_model_bi_obj = models_bi[best_lr_bi[0]]  
macro_bi = get_macro_avg_stats(best_model_bi_obj, val_loader)  
  
# --- 3. Prepare Stats for LSTM ---  
best_model_lstm_obj = models_lstm[best_lr_lstm[0]]  
macro_lstm = get_macro_avg_stats(best_model_lstm_obj, val_loader)  
  
# --- PRINTING THE REPORT ---  
  
print("\n" + "=" * 90)  
print("TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance")
```

```

print("=" * 90)
# Header for the manual table we are creating
print(f"{'Model':<20} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8} |  

      ↳{'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 90)

# 1. Simple RNN
t_simple = time.strftime('%H:%M:%S', time.  

    ↳gmtime(elapsed_simple[best_lr_simple[0]]))
print(f"{'Simple RNN':<20} | {best_lr_simple[0]:.0e} |  

      ↳{min(best_lr_simple[1]['val_loss']):.4f} | "  

      f"{'macro_simple['precision']:.4f} {'macro_simple['recall']:.4f} |  

      ↳{'macro_simple['f1-score']:.4f} | {t_simple}")

# 2. Bi-RNN
t_bi = time.strftime('%H:%M:%S', time.gmtime(elapsed_bi[best_lr_bi[0]]))
print(f"{'Bidirectional RNN':<20} | {best_lr_bi[0]:.0e} |  

      ↳{min(best_lr_bi[1]['val_loss']):.4f} | "  

      f"{'macro_bi['precision']:.4f} {'macro_bi['recall']:.4f} |  

      ↳{'macro_bi['f1-score']:.4f} | {t_bi}")

# 3. LSTM
t_lstm = time.strftime('%H:%M:%S', time.gmtime(elapsed_lstm[best_lr_lstm[0]]))
print(f"{'LSTM':<20} | {best_lr_lstm[0]:.0e} |  

      ↳{min(best_lr_lstm[1]['val_loss']):.4f} | "  

      f"{'macro_lstm['precision']:.4f} {'macro_lstm['recall']:.4f} |  

      ↳{'macro_lstm['f1-score']:.4f} | {t_lstm}")

```

TASK 3 SUMMARY: Best Learning Rates & Macro Avg Performance

Model	Best LR	Val Loss	Macro P	Macro R	Macro F1	Time
Simple RNN	8e-05	0.0423	0.7307	0.8723	0.7824	00:04:45
Bidirectional RNN	8e-05	0.0406	0.7421	0.8542	0.7861	00:08:42
LSTM	3e-04	0.0288	0.7846	0.9100	0.8346	00:05:03